Documentation de l'outil

Construction de tables d'expérience
Package `tablexp`, méthodes, vérifications et application

Une table d'expérience est l'estimation d'une transition d'un modèle multi-états

Établie le 17/10/2026 | Version 0.1 | Statut : Projet | Package `tablexp` 0.0.0.9000

Objet du document

Ce document est la documentation de l'**outil de construction de tables d'expérience** `outil_tables`. Il couvre le package R `tablexp`, qui porte toute la logique quantitative, les vérifications qui l'accompagnent (batteries de tests, contrôles du dépôt, plateforme de calcul) et, quand elle existera, l'application interactive qui l'exploite. Une *table d'expérience* est construite à partir des observations d'un portefeuille d'assurés : pour un risque donné, l'outil estime le taux d'une *transition* (vivant → décédé, en cours → racheté…) selon un ou plusieurs axes, le lisse ou le positionne sur une table de référence, le ferme aux âges extrêmes, le valide, et restitue la table avec tout ce qui permet de la rejouer et de la relire.

Les destinataires des tables et des rapports sont, par priorité, les hypothèses biométriques de la meilleure estimation (*best estimate*) Solvabilité II, que l'ACPR revoit en contrôle, puis la certification d'une table d'expérience par un actuaire indépendant agréé (`docs/exigences.md`, E1.2). Le document doit permettre à un relecteur externe, sans lire le code, de savoir ce qui a été calculé, avec quelles hypothèses, et de rejouer le calcul (E3.7). Les textes qui font foi sont recensés par le cahier des charges (`docs/exigences.md`, § 2.4) ; ils ne sont cités dans le présent document qu'une fois établis sur la source (issue #13).

Il est organisé en une partie par module du package (E3.1). La **partie I** présente l'outil : forme du package et graphe des modules, services transverses, reproductibilité, batteries de vérification et plateforme de calcul, contrôles du dépôt. La **partie II** décrit le point d'entrée unique et le plan de construction. Les parties suivantes suivent l'ordre des modules du moteur : données, exposition, taux bruts, lissage, positionnement, table prospective, fermeture, validation, rapport, puis le simulateur, qui produit les jeux de données sous une loi vraie connue. La dernière partie rassemble l'index des fonctions exportées, la traçabilité des exigences et les références. Chaque partie de module se termine par le tableau des fonctions du module, qui relie la description de la méthode à son implémentation ; un module qui n'existe pas encore a déjà sa partie, avec un tableau vide.

Le document ne porte aucun chiffre issu d'une exécution (E3.4) : les résultats d'une construction sont dans son rapport figé et sa fiche de construction, et les bilans des vérifications dans la sortie des batteries. Ses conventions de rédaction sont fixées par `docs/latex/CONVENTIONS.md` ; le vocabulaire est celui du glossaire du projet (`CONTEXT.md`).

Élément	Référence
Cahier des charges	`docs/exigences.md` (exigences E1.1 à E5.5)
Décisions d'architecture	`docs/adr/` (ADR 0002 à 0008)
Moteur de calcul	package `tablexp` (`R/`), packages de priorité `base` seulement (ADR 0002)
Interface	`app/app.R`, `app/affichage.R` (à venir ; aucun calcul)
Version de R	4.3.3 ; `DESCRIPTION` déclare `Depends: R (>= 4.3.3)`
Plateforme de référence	conteneur `rocker/r-ver:4.3.3` épinglé par empreinte, BLAS et LAPACK de référence (ADR 0008 ; section 4.3)
Données	aucune donnée réelle : jeux simulés sous une loi vraie et une graine

Sommaire

Partie I — Présentation de l'outil 6

1 Architecture du package et graphe des modules 6
1.1 Principe de séparation . . . 6
1.2 Fichiers du package . . . 6
1.3 Graphe d'appels autorisés . . . 7

2 Services transverses : `R/outils.R` 10
2.1 Refus : une condition R typée . . . 10
2.2 Avertissements métier . . . 10
2.3 Fonctions du module . . . 10

3 Reproductibilité et aléa 11
3.1 Exigence et portée . . . 11
3.2 Générateur fixé . . . 11
3.3 Exécution sous graine locale . . . 12
3.4 Vérifications . . . 12

4 Batteries de vérification et plateforme de CI 13
4.1 Trois batteries . . . 13
4.2 Harnais . . . 14
4.3 Plateforme de référence . . . 14
4.4 Jobs de la CI . . . 16

5 Contrôles du dépôt 16
5.1 Scripts de contrôle . . . 16
5.2 Absence de Java . . . 17

Partie II — Point d'entrée et plan de construction 19

6 Point d'entrée unique : `R/construire_table.R` 19
6.1 Plan de construction et validation . . . 19
6.2 Exécution . . . 20
6.3 Objet résultat . . . 20
6.4 Vérifications . . . 21
6.5 Fonctions du module . . . 21

Partie III — Module `donnees` 22

7 Données, rôles et contrôles : `R/donnees.R` 22
7.1 Conventions de temps . . . 22
7.2 Catalogue des rôles . . . 23
7.3 Contrôles, refus et exclusions . . . 24
7.4 Unité d'observation et fusion par tête . . . 26

7.5 Table de référence . . . 27
7.6 Vérifications . . . 28
7.7 Fonctions du module . . . 28

Partie IV — Module `exposition` 30

8 Exposition et événements : `R/exposition.R` 30
8.1 Recette d'exposition et conventions d'âge . . . 30
8.2 Troncature à gauche et censure à droite . . . 31
8.3 Découpage par année . . . 33
8.4 Vérifications . . . 33
8.5 Fonctions du module . . . 33

Partie V — Module `brut` 35

9 Taux bruts : `R/brut.R` 35
9.1 Fonctions du module . . . 35

Partie VI — Module `lissage` 36

10 Lissage : `R/lissage.R` 36
10.1 Fonctions du module . . . 36

Partie VII — Module `positionnement` 37

11 Positionnement sur une table de référence : `R/positionnement.R` 37
11.1 Fonctions du module . . . 37

Partie VIII — Module `prospective` 38

12 Tables prospectives : `R/prospective.R` 38
12.1 Fonctions du module . . . 38

Partie IX — Module `fermeture` 39

13 Fermeture aux âges extrêmes : `R/fermeture.R` 39
13.1 Fonctions du module . . . 39

Partie X — Module `validation` 40

14 Validation et suivi d'expérience : `R/validation.R` 40
14.1 Fonctions du module . . . 40

Partie XI — Module `rapport` 41

15 Fiche de construction et empreintes : `R/rapport.R` 41
15.1 Fiche de construction . . . 41
15.2 Empreinte des données . . . 42
15.3 Vérifications . . . 43
15.4 Fonctions du module . . . 43

Partie XII — Module `simulation` 45

16 Simulateur : `R/simulation.R` 45
16.1 Loi vraie et tables synthétiques . . . 45
16.2 Portefeuille simulé . . . 46
16.3 Constats de simulation . . . 48
16.4 Vérifications . . . 49
16.5 Fonctions du module . . . 49

Partie XIII — Index et traçabilité 51

17 Index et traçabilité 51
17.1 Index des fonctions exportées . . . 51
17.2 Traçabilité des exigences . . . 51
17.3 Références . . . 52

Partie I

Présentation de l'outil

Architecture du package et graphe des modules

1.1 Principe de séparation

L'outil sépare strictement le *calcul* de l'*affichage*. La règle est la suivante : si un résultat peut être calculé indépendamment de l'interface, son calcul figure dans le package `tablexp` (E4.3). Le moteur reçoit un `data.frame` et la déclaration des *rôles* de ses colonnes, jamais un fichier ; la lecture des fichiers, et les conversions de type qui l'accompagnent, relèvent d'une couche mince hors du moteur, qui n'existe pas encore (E4.4). Le moteur refuse un type incompatible, il ne le convertit pas.

Emplacement	Contenu	Interdit
`R/` (package `tablexp`)	toute la logique quantitative : rôles et validation des données, recettes d'exposition, estimation, lissage, positionnement, prospective, fermeture, validation, simulation, quantités des rapports et empreintes	toute primitive Shiny (`input$`, `reactive`, `render*`) ; tout package hors priorité `base` sans ADR de méthode ; toute dépendance à Java
`app/app.R` (à venir)	déclaration des rôles, édition du plan, appel du point d'entrée sur clic du bouton « Relancer les calculs », affichage	tout calcul ; tout appel d'une fonction non exportée (`tablexp:::`)
`app/affichage.R` (à venir)	formatage, tableaux HTML, tracés à partir des quantités du résultat	tout calcul
`tests/`	batteries, harnais, scripts de contrôle	—

L'application vit hors du package, sous `app/`, et non dans un fichier `R/affichage.R` : tout fichier de `R/` est installé dans l'espace de noms du package (section 1.2), si bien qu'un fichier d'affichage placé là entrerait dans le moteur et rendrait circulaire le contrôle de frontière (ADR 0007, point 1.3). Le dossier `app/` et son script de frontière n'existent pas encore : ils viennent avec la branche de l'application.

1.2 Fichiers du package

Le package `tablexp` est à la racine du dépôt (ADR 0007, point 1.1) :

— `DESCRIPTION` : `Package: tablexp`, `Depends: R (>= 4.3.3)` (décision M44), aucun champ `Imports`, `Suggests` ni `LinkingTo`, `Encoding: UTF-8`, `License: file LICENSE` ; le contrôle de la liste blanche (section 5) en fait une règle vérifiée ;

— `NAMESPACE`, écrit à la main : une directive `export` par fonction, sans `exportPattern` ; les fonctions exportées sont les points d'entrée et les fonctions que l'application appelle (contrôles de saisie à sens métier, constructeurs d'objets partagés), recensées dans l'index (section 17.1) ; aucune méthode S3 n'est exportée ;

— `R/` : un fichier par module et deux fichiers transversaux (tableau 1) ;

— `.Rbuildignore` : exclut de l'archive du package `app/`, `docs/`, `tests/`, `.claude/`, `.github/` et les fichiers de pilotage du dépôt.

Aucun fichier `Rd` ni `roxygen2` : la documentation de référence est le présent document, tenu concordant avec le code par la batterie de concordance (section 4 ; E3.3).

`R/` ne contient que du code du package. Une installation depuis l'arbre de travail (`R CMD INSTALL <dossier>`), qui est celle des batteries, n'applique pas `.Rbuildignore` : tout fichier de `R/` est installé, même s'il y est exclu (constat : un fichier `R/b.R` exclu par `.Rbuildignore` est présent dans le package installé ; ADR 0007, mesures). Un fichier étranger au moteur ne peut donc pas être rangé dans `R/`.

Table 1 : Fichiers de `R/`, dans l'ordre des modules (ADR 0007, points 1.2 et 2.1).

Fichier	Contenu	État
`R/outils.R`	services transverses sans contenu métier : graine locale, refus et avertissements (section 2) ; n'appelle aucun module	présent
`R/donnees.R`	rôles, contrôles, refus et exclusions, unité d'observation, table de référence (section 7)	présent
`R/exposition.R`	recettes d'exposition, conventions d'âge, troncature et censure (section 8)	présent
`R/brut.R`	taux bruts et intervalles (section 9)	à créer
`R/lissage.R`	lissages (section 10)	à créer
`R/positionnement.R`	positionnement sur une table de référence (section 11)	à créer
`R/prospective.R`	tables prospectives (section 12)	à créer
`R/fermeture.R`	fermeture aux âges extrêmes (section 13)	à créer
`R/validation.R`	validation et suivi d'expérience (section 14)	à créer
`R/rapport.R`	quantités des rapports, fiche de construction, empreintes (section 15)	présent
`R/simulation.R`	simulateur (section 16) ; hors de la chaîne du moteur	présent
`R/construire_table.R`	point d'entrée unique, validation du plan, orchestration seule, aucun calcul (section 6)	présent

1.3 Graphe d'appels autorisés

Les modules sont ordonnés : `outils`, `donnees`, `exposition`, `brut`, `lissage`, `positionnement`, `prospective`, `fermeture`, `validation`, `rapport`. Un module ne peut appeler que des fonctions définies dans les modules situés **strictement avant lui** dans cet ordre (ADR 0007, point 2.1). Deux fichiers sont hors de la chaîne : `simulation`, qui ne peut appeler que `donnees` et `outils` et qu'**aucun module du moteur n'appelle**, et `construire_table`, qui peut appeler tous les modules **sauf** `simulation`. La figure 1 et le tableau 2 donnent ce graphe.

Deux raisons fondent cette règle. Un ordre sans cycle rend chaque module lisible et vérifiable avec ses seuls modules amont : une anomalie d'un module ne peut atteindre que les modules situés après lui. Et le simulateur, qui produit les jeux sur lesquels le moteur est testé, ne partage pas le code qu'il sert à tester :

s'il pouvait appeler `exposition`, une erreur de la recette d'exposition se retrouverait à l'identique dans les données simulées et dans leur estimation, et la vérification ne la verrait pas (ADR 0007, « Options écartées »). Le simulateur reproduit le même modèle, pas le même code. Une seule exception à la lecture « en aval seulement » est assumée : `positionnement` peut appeler `lissage`, parce que la méthode 4 de positionnement est un lissage de la table du moment avec dérive de la référence (E2.12) ; le sens inverse est interdit, `lissage` n'appelant jamais `positionnement`.

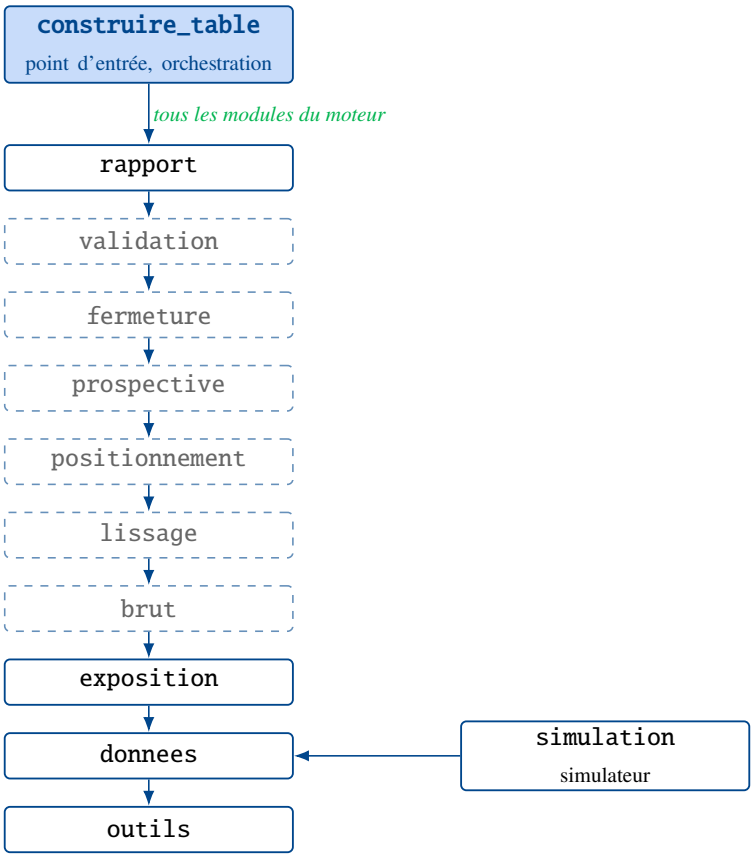


Figure 1 : Graphe d'appels autorisés des modules (ADR 0007, point 2.1). Une flèche de A vers B signifie que A peut appeler B et, la règle étant « strictement avant », tout module situé après B dans le sens des flèches. Blocs pleins : fichiers présents dans `R/` ; blocs en pointillé : modules prévus, fichiers absents. `simulation` n'est appelé par aucun module du moteur, ni par `construire_table` ; `lissage` n'appelle jamais `positionnement`.

Table 2 : Appels autorisés, par module appelant (ADR 0007, point 2.1 ; constante `APPELS_AUTORISES` de `tests/controles/graphe_modules.R`).

Module appelant	Peut appeler
`outils`	aucun module (fonctions des packages de priorité `base` seulement)
`donnees`	`outils`
`exposition`	`donnees`, `outils`
`brut`	`exposition` et les modules amont
`lissage`	`brut` et les modules amont
`positionnement`	`lissage` et les modules amont
`prospective`	`positionnement` et les modules amont
`fermeture`	`prospective` et les modules amont
`validation`	`fermeture` et les modules amont
`rapport`	tous les modules ci-dessus

Module appelant	Peut appeler
`simulation`	`donnees` et `outils` seulement
`construire_table`	tous les modules sauf `simulation`

Contrôle par script

Le graphe est vérifié par `tests/controles/graphe_modules.R`, en R de base (`codetools`, de priorité `recommended`, est exclu par l'ADR 0002), dans le job « Tests (R 4.3.3) » de la CI (ADR 0007, point 2.2) :

1. chaque fichier `R/<module>.R` est lu par `parse()` ; un fichier dont le nom n'est pas un module connu est un écart ;
2. les affectations de premier niveau (`nom <- ...` ou `nom = ...`, fonctions comme constantes) donnent la table « symbole → module » ; un symbole défini dans deux modules est un écart ;
3. pour chaque définition, les noms qu'elle emploie (`all.names()`), croisés avec cette table, donnent les arêtes entre modules ; toute arête absente du tableau 2 est un écart, imprimé avec le module appelant, le module appelé et les deux symboles ;
4. les appels dynamiques interdits (ci-dessous) sont cherchés dans tout l'arbre syntaxique.

Le script imprime toutes les arêtes trouvées et tous les écarts, puis sort avec le code 1 s'il en a trouvé au moins un. Ses limites sont écrites dans son en-tête : un nom local homonyme d'un symbole d'un autre module compte comme une arête (faux positif possible, jamais d'arête manquée par homonymie) ; un code de premier niveau qui n'est pas une affectation est un écart ; les appels vers les packages de base ne sont pas contrôlés ici, mais par la liste blanche de `DESCRIPTION`. Le script est lui-même testé sur des témoins (`tests/unitaires/test_controles.R`) : il échoue, entre autres, quand `exposition` appelle `brut`, quand `donnees` appelle `simulation`, quand `lissage` appelle `positionnement` et quand `construire_table` appelle `simulation`, et il passe sur un graphe conforme où `positionnement` appelle `lissage`.

Appels dynamiques interdits

Un appel dont la cible est calculée à l'exécution (`do.call("f", ...)`, `get("f")`) n'apparaît pas dans `all.names()` : il créerait une arête invisible pour le contrôle. Le script interdit donc dans `R/` (ADR 0007, point 2.3, liste élargie à la mise en œuvre et consignée dans l'annotation du 16/10/2026) :

— `do.call()` dont l'argument `what`, apparié par `match.call()` (position ou nom), n'est ni un symbole ni une fonction anonyme (chaîne, `paste0(...)...`) ; `do.call(f, args)` avec `f` un symbole désignant une fonction reste permis ;

— toute occurrence, appel ou fonction passée comme objet (`lapply(x, get)`), de `match.fun`, `get`, `get0`, `mget`, `getFromNamespace`, `getExportedValue`, `getFunction`, `asNamespace` et `getNamespace` ;

— `eval()` ou `evalq()` d'un `parse()`, `str2lang()`, `str2expression()`, `call()` ou `as.call()` ;

— l'opérateur `:::` et toute référence qualifiée `tablexp::` ;

— les formes qualifiées `base::`, `utils::` et `methods::` de ce qui précède, lues comme la forme nue.

Une fonction passée en argument puis appelée dans le corps de la fonction qui la reçoit crée une arête que le script attribue à l'appelant qui l'a passée : c'est voulu, la dépendance existe bien là. Les tests, seuls, appellent les fonctions non exportées par `tablexp:::` (ADR 0007, point 11.3).

Services transverses : R/outils.R

R/outils.R porte les services sans contenu métier partagés par tous les modules : l'exécution sous graine locale (section 3), la forme des refus et celle des avertissements (ADR 0007, point 1.2). Il n'appelle aucun module. Sans lui, la fonction de graine locale serait dupliquée dans le simulateur et dans le futur bootstrap par têtes, et chaque module inventerait sa forme de refus.

2.1 Refus : une condition R typée

Le moteur ne devine pas : quand une entrée ne permet pas de construire, il *refuse* (E2.3, E2.7). Un refus est une **condition R typée, jamais un objet retourné** (ADR 0007, point 6.4). `refus_tablexp()` construit la condition, de classe `c("refus_tablexp", "error", "condition")`, avec quatre champs :

— `module` : le module qui refuse ;

— `motif` : un code court stable, en minuscules ASCII, chiffres et soulignés, conforme au motif `MOTIF_CODE_REFUS` (`^[a-z][a-z0-9_]*$`) ;

— `message` : la phrase qui dit ce qui manque (E5.5) ;

— `details` : une liste (rôle, colonne, étape, paramètre, décompte, premiers numéros de ligne).

`refuser()` la lève par `stop()` et ne rend jamais la main. Un argument mal formé de `refus_tablexp()` (module ou message vide, motif non conforme, détails qui ne sont pas une liste) n'est pas un refus mais une erreur de programmation du package, signalée par `stop()`. Les tests vérifient le motif, le décompte et les numéros par `tryCatch()` ; l'application attrapera la condition et affichera son message. Une condition est préférée à un objet portant `ok = FALSE`, forme d'`outil_USP` : un tel objet peut être lu par mégarde comme une construction partielle, une condition ne le peut pas. Les codes de refus de chaque module sont recensés dans la partie du module.

2.2 Avertissements métier

Un avertissement métier signale ce qui n'empêche pas la construction mais doit être lu : unité d'observation par défaut, exclusion appliquée, relevé d'incohérence non bloquant. Il est **inscrit dans l'objet résultat, jamais émis par `warning()`** (ADR 0007, point 6.5) : l'objet est la seule source du rapport figé et de l'application ; un `warning()` serait perdu par l'application, absent de la fiche et parasite dans les batteries. `avertissements_vides()` crée l'accumulateur, un `data.frame` de colonnes `etape`, `code`, `message` et `nombre` ; `avertir()` lui ajoute une ligne, et refuse, par une erreur de programmation, un code qui n'est pas conforme à `MOTIF_CODE_REFUS`. Chaque fonction d'étape rend ses avertissements ; le point d'entrée les rassemble dans le champ `avertissements` de l'objet et de la fiche. Les codes de chaque étape sont recensés dans la partie de son module (sections 7.4 et 8).

2.3 Fonctions du module

Table 3 : Définitions de premier niveau de `R/outils.R`.

Nom	Statut	Entrées	Sortie et rôle
`GENERATEUR`	constante	—	générateur fixé (section 3.2) : Mersenne-Twister, inversion pour la loi normale, rejet pour `sample()`
`MOTIF_CODE_REFUS`	constante	—	motif d'un code de refus ou d'avertissement : `^[a-z][a-z0-9_]*$`
`refus_tablexp()`	interne	`module`, `motif`, `message`, `details` (liste, vide par défaut)	condition de classe `refus_tablexp`, sans la lever ; `stop()` si un argument est mal formé (section 2.1)
`refuser()`	interne	mêmes arguments	lève le refus ; ne rend jamais la main
`avertissements_vides()`	interne	—	accumulateur vide des avertissements métier (section 2.2 ; issue #9)
`avertir()`	interne	accumulateur, `etape`, `code`, `message`, `nombre`	accumulateur augmenté d'une ligne, sans `warning()` (section 2.2 ; issue #9)
`sous_graine()`	interne	`graine` (entier explicite, sans défaut), `expr`	valeur de `expr` évaluée sous la graine et `GENERATEUR`, état du générateur de l'appelant restauré ; refus `graine_invalide` (section 3.3)

Reproductibilité et aléa

3.1 Exigence et portée

À données, rôles, plan et graine identiques, le programme produit des résultats identiques **au bit près sur une même machine** ; toute simulation passe par une graine locale explicite sous un générateur fixé, consignée dans le résultat, l'état global du générateur étant restauré à la sortie ; aucun horodatage n'entre dans la partie du résultat comparée lors d'un rejeu (E4.6). La présente section décrit les deux mécanismes du socle : le générateur fixé et l'exécution sous graine locale. La consignation de la graine et du générateur dans la fiche de construction et le rejeu de la fiche sont décrits avec la fiche (section 15.1).

> **Statut.** L'identité au bit près est un *constat* des batteries (section 3.4), établi sur la machine qui les exécute ; ce n'est pas une propriété démontrée. L'identité entre deux machines n'est **pas garantie** : deux processeurs différents pourraient différer au dernier bit, même dans le même conteneur, si la bibliothèque mathématique du système choisit ses variantes de calcul selon le processeur (hypothèse de l'ADR 0008, non mesurée). C'est pourquoi les références de non-régression seront comparées à tolérance et produites sur une plateforme unique (section 4.3).

3.2 Générateur fixé

Le générateur est la constante `GENERATEUR` de `R/outils.R` : `kind = "Mersenne-Twister"`, `normal.kind = "Inversion"`, `sample.kind = "Rejection"`. Ce sont les valeurs par défaut de R 4.3.3 (`RNGkind()` dans une session neuve), fixées en constante pour qu'un changement de défaut d'une version de R ne change pas les tirages en silence. Le générateur uniforme est le Mersenne-Twister de

Matsumoto et Nishimura ; les lois normales s'obtiennent par inversion de la fonction de répartition ; la génération uniforme discrète (`sample()`) emploie la méthode par rejet, défaut de R depuis la version 3.6.0, la méthode antérieure par arrondi rendant `sample()` sensiblement non uniforme sur de grandes populations (documentation de R, `?Random`).

Référence. M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`).

Un changement de `GENERATEUR` change tout résultat simulé : c'est un changement de résultat, soumis au visa (ADR 0007, « Conséquences »).

3.3 Exécution sous graine locale

`sous_graine(graine, expr)` est la seule voie par laquelle le package tire un aléa (ADR 0007, point 8.1). Elle procède ainsi :

1. **contrôle de la graine** : un nombre unique, non manquant, fini, entier et de valeur absolue au plus `.Machine$integer.max` ; sinon, et aussi quand la graine est omise, refus `graine_invalide` du module `outils`. Il n'y a jamais de graine par défaut : une graine par défaut serait une graine non déclarée ;
2. **sauvegarde de l'état de l'appelant** : `.Random.seed` de l'environnement global, ou son absence, et le réglage `RNGkind()` ;
3. **tirage** : `set.seed(graine, kind, normal.kind, sample.kind)` sous `GENERATEUR`, puis évaluation de `expr` ;
4. **restauration**, par `on.exit()`, donc aussi après une erreur dans `expr` : le réglage d'abord, parce que `RNGkind()` réécrit `.Random.seed`, puis `.Random.seed` de l'appelant, ou sa suppression s'il n'existait pas.

Les tirages ne dépendent donc que de la graine : ni de l'état du générateur de l'appelant, ni de son réglage, que `sous_graine()` laisse inchangés. Le simulateur (section 16) est, à cette date, la seule fonction du package qui tire un aléa ; il reçoit sa graine en argument et la consigne, avec `GENERATEUR`, dans le jeu simulé.

Limite. La réserve de la méthode normale de Box-Muller d'un appelant, gardée par R hors de `.Random.seed`, n'est pas restaurable ; c'est sans effet sur le moteur, qui tire toujours sous `Inversion` (limite reprise d'`outil_USP`, ADR 0007, point 8.1).

Aucun aléa hors calcul. Les fichiers temporaires, employés par exemple pour calculer une empreinte par `tools::md5sum()`, sont créés par `tempfile()`, qui ne touche pas `.Random.seed` (constat : dans une session neuve, `.Random.seed` n'existe ni avant ni après `tempfile()` et `tools::md5sum()` ; ADR 0007, point 8.2).

3.4 Vérifications

Deux fichiers de tests portent ces propriétés (section 4) :

— `tests/unitaires/test_outils.R` : valeur de `GENERATEUR` ; mêmes tirages que `set.seed()` nu sous le générateur par défaut ; graine entière écrite en double admise ; appelant sous `L'Ecuyer-CMRG` : tirages de `GENERATEUR`, `.Random.seed` et `RNGkind()` de l'appelant restaurés ; absence de `.Random.seed` restaurée ; état restauré après une erreur dans `expr` ; refus `graine_invalide` des graines mal formées ;

— `tests/test_reproductibilite.R` : à graine égale, tirages identiques (`identical()`) dans un même processus, quel que soit l'état du générateur de l'appelant, et dans un autre processus R de la même machine ; à graines différentes, tirages différents. Les tirages comparés couvrent les trois réglages du

générateur (`runif()`, `rnorm()`, `sample.int()`) ; le processus fils les écrit en `%.17g`, format qui restitue un double au bit près. La même batterie vérifie le simulateur et la construction (sections 16 et 15).

Ces vérifications sont des constats sur la machine qui exécute la batterie ; sur la CI, c'est la plateforme de référence (section 4.3).

Batteries de vérification et plateforme de CI

4.1 Trois batteries

Les vérifications du code passent par trois batteries en R de base, sans `testthat` (package du CRAN, hors priorité `base`), lancées depuis la racine du dépôt (ADR 0007, point 11.1) :

```
Rscript tests/test_unitaires.R
Rscript tests/test_reproductibilite.R
Rscript tests/concordance_doc_moteur.R --strict
```

La même liste, dans le même ordre, figure dans `CLAUDE.md` (« Commandes »), dans le workflow `.claude/workflows/circuit-technique.js` et dans le job « Tests (R 4.3.3) » de la CI ; un script le contrôle (section 5). Chaque batterie :

— **installe l'arbre de travail** dans une bibliothèque temporaire (`R CMD INSTALL --no-docs -l <bibliothèque> <racine>`, par `installer_tablexp()`), sans installer aucun autre package, puis charge `tablexp` depuis elle et échoue s'il est chargé d'ailleurs : c'est le code de l'arbre qui est testé, jamais un package installé auparavant ;

— affiche `R.version.string` puis, à la fin, une **ligne de bilan** : version de R, nombre de tests, d'échecs, d'échecs attendus et de succès inattendus ;

— **sort avec un code non nul** en cas d'échec ou de succès inattendu (section 4.2).

Une bibliothèque déjà installée, désignée par la variable d'environnement `TABLEXP_BIBLIOTHEQUE`, n'est réutilisée que si l'*empreinte de l'arbre source* inscrite à son installation est égale à celle de l'arbre courant ; sinon la batterie réinstalle et le dit. Cette empreinte est un MD5 (`tools::md5sum()`) de `DESCRIPTION`, de `NAMESPACE` et, s'ils existent, de tous les fichiers de `R/`, `inst/`, `data/` et `src/`, chemins relatifs compris, triés par octets (`empreinte_source()`). Elle empêche de tester un package périmé ; elle n'a aucun rôle dans les résultats.

`tests/test_unitaires.R` exécute chaque fichier `tests/unitaires/test_*.R`, triés par nom, chacun dans son propre environnement ; `quit()`, `q()` et `bilan()` y sont neutralisés, une erreur au chargement d'un fichier est un échec, et un fichier sans aucune assertion aussi. **`tests/test_reproductibilite.R`** vérifie les identités de la section 3.4, puis celles du simulateur et de la construction ; la comparaison aux références de non-régression s'y ajoutera avec la branche qui les crée. **`tests/concordance_doc_moteur.R`** confronte le présent document au code installé : toute fonction citée existe ; toute fonction exportée est citée, ou inscrite dans la liste d'attente `tests/concordance_attente.csv` avec renvoi à l'issue qui la documentera, où elle compte en échec attendu ; chaque catalogue recensé par le document (fonctions exportées, définitions de premier niveau de chaque fichier de `R/`, rôles, recettes d'exposition) est égal, nom pour nom, à sa source dans le code ; toute ligne de la liste d'attente désigne un export ou un catalogue requis et renvoie à une issue. Le format des citations et des catalogues qu'elle lit est fixé par `docs/latex/CONVENTIONS.md`

(section 3) ; les tableaux de fonctions de chaque module en sont la principale application (E4.9). Sans `--strict`, elle rend compte des écarts sans échouer.

4.2 Harnais

Les trois batteries partagent `tests/harnais.R`, repris de la forme d'`outil_USP` (ADR 0007, point 11.2, et annotation du 16/10/2026, point d) :

— `verifier(nom, expr)` : succès si `expr` vaut `TRUE` ; une chaîne décrit l'écart ; toute autre valeur, ou une erreur, est un échec ;

— `echec_attendu(nom, issue, expr, motif, erreur_quelconque)` : **défaut connu**, codé en échec attendu avec renvoi obligatoire à l'issue, de la forme `#N` ; `expr` décrit le comportement *correct*. Tant qu'elle échoue, le test est « attendu » ; **si elle passe, c'est un succès inattendu et la batterie échoue** : le défaut semble corrigé, la marque est retirée et le test devient ordinaire. Trois natures d'échec attendu sont admises et décomptées : par valeur (`expr` rend `FALSE` ou une chaîne), par refus (un `refus_tablexp`, du motif donné s'il l'est) et, seulement si `erreur_quelconque = TRUE`, par une autre erreur ; toute autre issue est un échec, pour qu'un défaut connu ne masque pas une panne différente ;

— `proche(a, b, rel, abs)` : égalité numérique à tolérance déclarée, $|a - b| \leq abs + rel\,|b|$ élément par élément, sans tolérance par défaut ; rend un écart pour des valeurs non numériques, des longueurs différentes ou des valeurs manquantes à des positions différentes ; exige l'égalité exacte des valeurs infinies ;

— `leve_refus(expr, motif)` : vrai si `expr` lève un `refus_tablexp` du motif donné ;

— `sans_execution(x)` : retire de l'objet résultat le champ des informations d'exécution, seul champ qui change entre deux exécutions identiques, avant `identical()` ;

— `bilan()` : ligne de bilan et code de sortie. Il compte en outre un échec si un espace de noms hors priorité `base`, autre que `tablexp`, est chargé en fin de batterie (`espaces_hors_base()`) : l'image de la CI contient des packages `recommended`, et cette règle empêche un test de s'appuyer sur l'un d'eux à l'insu de la liste blanche (ADR 0008, « Conséquences »).

Une **garde de sortie** (`.Last()`) fait sortir avec le code 1 tout processus qui a chargé le harnais et se termine avant la ligne de bilan, même par `quit(status = 0)`. Le harnais est lui-même testé (`tests/unitaires/test_harnais.R`) : une assertion fausse donne un code de sortie non nul, et un échec attendu qui réussit fait échouer la batterie.

Chaque test nomme la nature de son attendu (ADR 0007, point 11.4 ; E2.20) : valeur calculée à la main (identité exacte, ou tolérance déclarée) ; *recoupement*, comparaison à la valeur que rend, dans la batterie, un appel direct d'une fonction d'un package de priorité `base`, dont la portée est limitée quand le moteur repose sur la même fonction : le test dit alors ce qu'il vérifie (la formule et son branchement, non la fonction de base) ; *contre-vérification*, valeur de référence produite hors CI par un package avancé et écrite en dur avec sa version et sa date ; *constat de simulation*, qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (`CONTEXT.md` ; E2.19).

4.3 Plateforme de référence

Les références de non-régression, qui viendront avec la deuxième branche, sont produites et comparées sur une *plateforme de référence* unique (E4.7 ; ADR 0008) : le job de la CI s'exécute dans un **conteneur `rocker/r-ver:4.3.3` épinglé par empreinte**, dont R charge la **BLAS et la LAPACK de référence** (Netlib), en monothread. Identifiant fixé par l'annotation du 16/10/2026 de l'ADR 0008 (décision M47) :

image rocker/r-ver:4.3.3@sha256:
732d15020af326da9e919c07f70ca32bf5d3e409220af32e0a4b6d0a89437309
suivie de la bascule update-alternatives vers les paquets libblas3 et liblapack3 3.10.0-2ubuntu1 d'Ubuntu 22.04 ;
R version 4.3.3 (2024-02-29) ;
BLAS /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0 ;
LAPACK /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0, version 3.10.0 ;
monothread (OPENBLAS_NUM_THREADS et OMP_NUM_THREADS valent 1).

Pourquoi un conteneur épinglé. Un libellé d'exécuteur (ubuntu-24.04) n'est pas une image figée : elle est mise à jour par GitHub, bibliothèques comprises, sans commit dans le dépôt. outil_USP, qui installait R sur l'image de l'exécuteur, a constaté une dérive entre deux exécutions de même R et de même image, due à une BLAS optimisée (OpenBLAS) qui choisit ses noyaux selon le processeur (constat d'outil_USP rapporté par l'ADR 0008, « Contexte »). Une étiquette de conteneur peut elle aussi désigner une autre image après une reconstruction (hypothèse de l'ADR 0008, non vérifiée) ; l'empreinte (@sha256:...) désigne un contenu immuable. Le conteneur est une fonctionnalité native de GitHub Actions, et non une action tierce, ce que la politique des Actions du dépôt exige.

Pourquoi la bascule. L'image charge par défaut OpenBLAS (relevé de la première exécution, rapporté par l'annotation du 16/10/2026 de l'ADR 0008). Le mainteneur a retenu de basculer dans le job vers la Netlib déjà présente dans l'image, plutôt que de compiler R depuis la source (décision M47 ; le repli par compilation reste prévu par l'ADR 0008, point 10). L'étape « BLAS et LAPACK de référence » de .github/workflows/ci.yml pointe par update-alternatives les alternatives libblas.so.3-x86_64-linux-gnu et liblapack.so.3-x86_64-linux-gnu vers libblas3 et liblapack3, puis affiche les cibles résolues et les versions des paquets. La plateforme de référence est l'image épinglée *suivie de cette bascule* : modifier l'étape, l'image ou les bibliothèques est un changement de plateforme, décidé par le mainteneur et, une fois des références existantes, accompagné d'un tableau avant / après de toutes les références (ADR 0008, point 9).

Script de plateforme. tests/controles/plateforme.R s'exécute en première étape de chaque job R qui calcule, avant tout calcul (ADR 0008, points 3 à 5) ; le job « Absence de Java », qui n'effectue aucun calcul numérique et ne produit aucun résultat, en est exempté, ainsi que des variables monothread (ADR 0008, annotation du 17/10/2026). En mode strict (--exiger, sur la CI), il **échoue** si : R.version.string n'est pas exactement R_ATTENDU (R version 4.3.3 (2024-02-29)) ; l'une des quatre lectures de la BLAS et de la LAPACK (extSoftVersion()["BLAS"], sessionInfo()$BLAS, sessionInfo()$LAPACK, La_library()) est vide, désigne une BLAS optimisée (motif MOTIF_OPTIMISE : openblas, mkl, atlas, blis, flexiblas) ou sort des chemins de référence (MOTIF_BLAS, MOTIF_LAPACK) ; La_version() n'est pas LA_VERSION_ATTENDUE (3.10.0) ; OPENBLAS_NUM_THREADS ou OMP_NUM_THREADS ne vaut pas 1 ; TABLEXP_IMAGE, que le job renseigne avec la même chaîne que l'image du conteneur, n'est pas épinglée par empreinte (MOTIF_IMAGE). Sans --exiger (poste, session), il affiche les mêmes grandeurs et les écarts sans échouer : les batteries restent exécutables sur le poste Windows du mainteneur, dont la BLAS est celle de R. Dans les deux modes, il écrit l'*identifiant de plateforme* (image, R.version.string, BLAS, LAPACK et sa version, sessionInfo()) et consigne sans faire échouer le processeur et ses drapeaux (fma, avx2, avx512f), ImageOS et ImageVersion de l'hôte, LC_COLLATE, le fuseau horaire, la présence d'un exécutable java et GITHUB_SHA. Ce bloc sera reporté dans le message de chaque commit de référence.

Ce que la plateforme ne garantit pas. Le modèle de processeur varie d'une exécution à l'autre sous le même libellé d'exécuteur (relevé de l'ADR 0008, annotation du 16/10/2026). La BLAS Netlib n'a pas de noyau choisi selon le processeur ; la réserve sur la bibliothèque mathématique du système (section 3.1) reste

en revanche ouverte, non mesurée. Les références seront donc comparées à tolérance, structure contrôlée à part (E4.7).

4.4 Jobs de la CI

`.github/workflows/ci.yml` définit les jobs suivants, exécutés sur chaque pull request, sur chaque push vers `main` et sur demande. Aucun job n'installe de package du CRAN ni ne configure de dépôt CRAN pour une installation : `tablexp` est installé depuis l'arbre de travail par chaque batterie.

Table 4 : Jobs de la CI (`.github/workflows/ci.yml`).

Job	Exécution	Contenu
« Contrôles du dépôt »	hôte `ubuntu-24.04`, sans R	réglages de Claude Code valides (JSON) ; fiches d'agents complètes ; fiches `-approfondi` à jour ; aucun fichier sensible versionné
« Tests (R 4.3.3) »	conteneur de la plateforme de référence	bascule vers la BLAS et la LAPACK de référence ; script de plateforme en mode strict ; les trois batteries ; liste blanche de `DESCRIPTION` ; ordre des modules ; identité des batteries et de l'image. Les étapes des batteries et des contrôles, sous la condition `${{ !cancelled() }}`, s'exécutent même si une étape antérieure échoue, pour que tous les écarts soient visibles ; le job reste en échec. L'étape du script de plateforme, sans condition, est sautée si la bascule vers la BLAS de référence échoue
« Absence de Java »	même conteneur, réseau requis	contrôle de l'absence de Java (section 5.2) et son cas témoin
« Documentation »	hôte `ubuntu-24.04`, sans R	installation des paquets TeX par `apt-get` (versions affichées), compilation du présent document selon `docs/latex/CONVENTIONS.md` (section 6 : passes jusqu'à stabilisation, cinq au plus ; échec sur une erreur ou un renvoi indéfini), PDF et journal publiés en artefact

Ces quatre jobs sont les contrôles requis que liste le ruleset de `main` (`.github/ruleset-main.json`, appliqué au dépôt par le mainteneur ; ADR 0008, point 6) : une pull request ne peut alors être fusionnée que s'ils sont verts. Le job « Documentation » ne produit aucun résultat ; le PDF versionné reste celui que commite la session principale, et la version des paquets TeX suit l'image de l'exécuteur. Les batteries et les scripts des jobs conteneurisés ne dépendent pas de git, absent de l'image ; un contrôle qui lit l'historique reste dans « Contrôles du dépôt », sur l'hôte (ADR 0008, point 11).

Contrôles du dépôt

5.1 Scripts de contrôle

Tout contrôle mécanique est un script, jamais une relecture (`CLAUDE.md`, « Workflows », principe 7). Les scripts sont en R de base, sous `tests/controles/`, et sortent avec un code non nul en cas d'écart.

Chacun est testé sur des témoins dans `tests/unitaires/test_controles.R` : il doit échouer sur un cas fautif et passer sur le dépôt.

Table 5 : Scripts de contrôle (`tests/controles/`).

Script	Règle contrôlée	Source
`liste_blanche_description.R`	`Depends` : exactement `R (>= 4.3.3)`, plus d'éventuels packages de priorité `base` ou admis ; `Imports` : packages de priorité `base` de R 4.3.3, ou admis ; `LinkingTo`, `Suggests`, `Enhances` : packages admis seulement. Un package est admis s'il figure, avec son champ et l'ADR de méthode qui l'introduit, dans la constante `ADMIS`, vide à cette date	ADR 0002, point 3 et A8 ; M44
`graphe_modules.R`	graphe d'appels autorisés et appels dynamiques interdits (section 1.3)	ADR 0007, points 2.1 à 2.3
`identite_batteries.R`	mêmes batteries, dans le même ordre, dans `CLAUDE.md`, dans `circuit-technique.js` et dans les étapes « Batterie : » du job « Tests (R 4.3.3) » ; ce job ne porte ni `continue-on-error`, ni condition `if:` de job, et une étape de batterie n'a pas d'autre condition que `${{ !cancelled() }}` ; une seule chaîne d'image dans tous les workflows	`CLAUDE.md` ; ADR 0008, point 1
`plateforme.R`	plateforme de référence (section 4.3)	ADR 0008, points 3 à 5
`absence_java.R`	aucune dépendance à Java (section 5.2)	ADR 0002, A1 ; ADR 0008, point 7 ; issue #3

La liste des packages de priorité `base` est écrite en dur dans `liste_blanche_description.R` pour R 4.3.3 (`PACKAGES_BASE_R433`) : le verdict ne dépend pas de la machine ; le script signale, sans échouer, un écart avec les packages de priorité `base` effectivement installés.

5.2 Absence de Java

Règle. Aucun code du projet ne dépend de Java, ni directement ni indirectement : ni le moteur, ni l'application, ni la contre-vérification (ADR 0002, annotation du 16/10/2026, A1, décision M40). Java n'est pas utilisable sur le poste de travail du mainteneur ; c'est pour cette raison que le package `ELT`, qui dépend de `xlsx` et donc de Java, est exclu de la contre-vérification. Sa source ne peut être exécutée qu'hors du dépôt, sans Java, en appoint, sans qu'aucun code en soit copié (ADR 0002, A5 et troisième annotation).

Contrôle. `tests/controles/absence_java.R` (issue #3) contrôle trois ensembles de packages :

— le *moteur* : les packages déclarés dans les champs `Depends`, `Imports` et `LinkingTo` de `DESCRIPTION`, hors R ; aucun à cette date ;

— l'*application* : `shiny`, `htmltools`, et `openxlsx` à titre facultatif ;

— la *contre-vérification* : les packages `survival`, `mgcv`, `WH`, `StMoMo`, `demography`, `MortalityLaws`, `lifecontingencies`, `survPen`, `MortCast`, `locfit`, et `flexsurv` à titre facultatif.

Les deux derniers ensembles sont lus dans une liste unique, le fichier `packages_hors_moteur.csv` du dossier `tests/controles/` (colonnes `ensemble`, `package`, `statut`, `source`), qui est la référence ; la liste

ci-dessus en est la transcription à la date du document. Le script lit les métadonnées d'un **instantané daté** du CRAN chez Posit Package Manager, par `available.packages()`, sans rien installer ; la date est la constante `INSTANTANE` du script (2026-10-15, un jour ouvré : selon la documentation de Posit citée par l'ADR 0008, aucun instantané n'est publié les week-ends et jours fériés), qui ne change que par un commit `repo:`. Il calcule, pour chaque package, la fermeture récursive de ses dépendances `Depends`, `Imports` et `LinkingTo`, par la fonction `tools::package_dependencies()` appelée avec `recursive = TRUE`, et **échoue** (code 1) :

— si un package de `PACKAGES_JAVA` (`rJava`, `xlsx`, `xlsxjars`) figure dans un ensemble ou dans la fermeture de l'un de ses packages ; la chaîne de dépendances la plus courte qui y mène est affichée ;

— si un package listé, ou une dépendance récursive hors priorité `base`, manque aux métadonnées : un package introuvable n'a pas de dépendances connues, il ne passe pas en silence ;

— si la liste est mal formée (en-tête, ensembles et statuts connus, noms valides, sans doublon, ensembles non vides).

Des métadonnées illisibles (panne réseau, adresse fausse) donnent le code 2 : aucun verdict n'est rendu, l'échec du job n'est jamais masqué et une relance le lève. Seul le filtre `duplicates` d'`available.packages()` est appliqué : le filtre par version de R écarterait les packages qui exigent un R plus récent que celui qui exécute le contrôle, alors que la contre-vérification peut tourner sur un autre poste muni d'un R récent, et ces packages seraient alors signalés absents (faux échec) ; le filtre par système d'exploitation est écarté aussi, le poste étant sous Windows et la CI sous Linux.

Cas témoin. Le job « Absence de Java » exécute aussi le script avec `--ajouter ELT` : il doit rendre le code 1 (un verdict, et non le code 2 d'une panne) et afficher la chaîne de `ELT` à `xlsx`. Les tests du script s'exécutent sans réseau, sur des dépôts locaux témoins (`tests/unitaires/test_controles.R`).

Limite. La détection porte sur les seuls noms de `PACKAGES_JAVA`. Le champ `SystemRequirements`, qui pourrait déclarer Java, n'est pas renseigné dans les métadonnées `PACKAGES` du CRAN (relevé consigné dans l'en-tête du script) : un package qui exigerait Java sans dépendre de l'un de ces noms échapperait au contrôle. Étendre la détection relève du mainteneur.

Partie II

Point d'entrée et plan de construction

Point d'entrée unique : R/construire_table.R

Le package a un point d'entrée unique, `construire_table(donnees, roles, plan, provenance = NULL)`, qui exécute un *plan de construction* d'un bloc et retourne un objet complet (ADR 0004 ; ADR 0005 ; ADR 0007, point 3.1). `R/construire_table.R` se limite à l'orchestration : il valide le plan, appelle une fonction d'étape par module dans l'ordre du plan et assemble l'objet, sans aucun calcul. Le quatrième argument, `provenance`, facultatif, est la voie par laquelle la couche de lecture transmet le nom du fichier lu, son empreinte et l'algorithme de celle-ci ; `NULL` signifie « données construites en mémoire, aucun fichier », ce que la fiche écrit (section 15.1).

6.1 Plan de construction et validation

Le plan est une liste R nommée, construite en mémoire par l'appelant, et **validée entièrement avant toute exécution, sans aucun défaut implicite** (ADR 0005, point 3 ; ADR 0007, point 5) : un paramètre absent est un refus, jamais une valeur supposée. Il porte six champs de premier niveau (constante `CHAMPS_PLAN` ; décisions M48 et M49) :

— `transition` : `list(depart = , arrivee = )`, deux noms d'états ; dans cette branche, `vivant` → `deces` ou `en_cours` → `rachat` ; les autres causes de sortie sont des sorties concurrentes ;

— `periode` : `list(debut = , fin = )`, deux dates, fin incluse ;

— `delai_declaration` : entier de jours ≥ 0, requis (zéro explicite admis) ;

— `segmentation` : vecteur de rôles pris dans `sexe` et `segment`, sans doublon, requis, vide explicite admis (E2.9). Elle est au premier niveau, et non parmi les paramètres de l'étape `exposition`, parce que l'étape `donnees`, qui précède, en a besoin pour ses contrôles (rôle requis, `segment_manquant`, `tete_segment_incoherent`) : une étape antérieure ne doit pas dépendre d'un paramètre d'une étape postérieure, et un plan réduit à l'étape `donnees` doit rester défini sans défaut (décision M49) ;

— `graine` : entier, requis dès qu'une étape du plan tire un aléa, interdit sinon. Aucune étape de cette branche ne tire d'aléa : la graine y est donc refusée, et la fiche écrit « aucune étape aléatoire » ;

— `etapes` : liste nommée ordonnée ; chaque nom est une étape, au plus une fois, dans l'ordre des modules, et chaque valeur la liste nommée des paramètres de l'étape, tous requis.

`transition`, `periode`, `delai_declaration` et `segmentation` sont déclarés une seule fois et transmis par le point d'entrée, en arguments nommés, à chaque fonction d'étape qui les emploie ; **une fonction d'étape ne reçoit jamais le plan entier** (ADR 0007, annotation du 17/10/2026, point c). Deux étapes ne peuvent

donc pas lire deux périodes ou deux segmentations différentes, et le chemin d'un champ dans le plan n'est connu que du point d'entrée et du validateur.

Les étapes de la branche et leurs paramètres sont des constantes du package (ETAPES, PARAMETRES_ETAPES), que le validateur applique :

Étape	Fonction d'étape	Préalable	Paramètres (tous requis)
donnees	preparer_donnees()	—	exclusions : vecteur de codes de motif sans doublon, vide explicite admis (section 7.3)
exposition	calculer_exposition()	donnees	convention : plus_fine, dernier_anniversaire ou age_atteint_annee ; par_annee : TRUE ou FALSE (section 8.1)

Ni l'une ni l'autre ne tire d'aléa. La recette d'exposition et l'unité d'observation ne sont pas des paramètres : le moteur les déduit des rôles (sections 8.1 et 7.4) ; seule une convention d'âge plus grossière peut être forcée.

valider_plan(plan), exportée parce que l'application l'appellera comme contrôle de saisie (E4.3), rend le plan inchangé, ou refuse. Elle porte les codes *structurels* du plan, module construire_table : plan_mal_forme, champ_inconnu, champ_manquant, etapes_mal_formees, aucune_etape, etape_en_double, etape_inconnue, etapes_hors_ordre, etape_prealable_manquante, graine_inattendue, graine_manquante, parametres_mal_formes, parametre_inconnu et parametre_manquant. Elle ne réimplémente pas les contrôles de valeur : elle appelle ceux des modules (controler_parametres_donnees(), controler_parametres_exposition()), dont les codes et le module sont ainsi identiques, quelle que soit la fonction qui détecte la condition (ADR 0007, annotation du 17/10/2026, point d).

Exemple de plan (vivant → décédé, par sexe, sans exclusion) :

```
plan <- list(transition = list(depart = "vivant", arrivee = "deces"),
             periode = list(debut = as.Date("2015-01-01"),
                            fin = as.Date("2024-12-31")),
             delai_declaration = 0L, segmentation = "sexe",
             etapes = list(donnees = list(exclusions = character(0)),
                           exposition = list(convention = "plus_fine",
                                             par_annee = FALSE)))
res <- construire_table(donnees, roles, plan)
```

6.2 Exécution

construire_table() procède dans cet ordre : validation du plan ; contrôle des rôles (controler_roles()) ; déduction de la recette et, si le plan comprend l'étape exposition, de la convention retenue ; contrôle de la provenance ; calcul de l'empreinte des données reçues (section 15.2). Tout refus possible avant le calcul a donc lieu avant la première étape. Les étapes s'exécutent ensuite dans l'ordre du plan, chacune par sa fonction d'étape, qui rend ses paramètres effectifs, son résultat, ses diagnostics et ses avertissements ; l'étape exposition lit la table des observations de l'étape donnees.

6.3 Objet résultat

L'objet retourné est une liste nommée de classe S3 construction (ADR 0007, point 6.1), lue par $ et jamais recalculée par ses lecteurs :

— table : la table finale ; NULL dans cette branche, dont les plans s'arrêtent à l'exposition ;

— `etapes` : une entrée par étape exécutée, dans l'ordre, avec `parametres` (paramètres effectifs, après résolution par le moteur), `resultat` et `diagnostics` ;

— `avertissements` : le `data.frame` (`etape`, `code`, `message`, `nombre`) des avertissements de toutes les étapes (section 2.2) ;

— `fiche` : la fiche de construction (section 15.1) ;

— `execution` : horodatage de début, durée, version de R, plateforme, système, paramètres régionaux et packages attachés ; **hors de toute comparaison** (rejeu, reproductibilité, non-régression) : c'est le seul champ qui change entre deux exécutions identiques.

Ajouter un champ est compatible ; renommer ou retirer un champ change le contrat partagé « objet résultat » (ADR 0007, point 6.3).

6.4 Vérifications

`tests/unitaires/test_construire_table.R` vérifie la liste des exports, chaque refus de `valider_plan()` (noms exacts des champs de transition et de période compris), la cohérence entre `ETAPES` et les fonctions d'étape appelées, la structure de l'objet et de la fiche, et des identités exactes sur un portefeuille simulé (section 8.4). `tests/test_reproductibilite.R` vérifie que deux constructions identiques sont `identical()` hors `execution`, dans le même processus et dans un autre, et que le rejeu de la fiche redonne le même objet.

6.5 Fonctions du module

Table 6 : Définitions de premier niveau de `R/construire_table.R`.

Nom	Statut	Entrées	Sortie et rôle
`CHAMPS_PLAN`	constante	—	les six champs de premier niveau du plan
`CHAMPS_PLAN_REQUIS`	constante	—	les cinq champs toujours requis (tous sauf `graine`)
`ETAPES`	constante	—	étapes : module, fonction d'étape, préalable, tirage d'un aléa
`PARAMETRES_ETAPES`	constante	—	paramètres de chaque étape : nom, type, domaine
`liste_nommee()`	interne	objet	liste nommée aux noms uniques et non vides
`valider_plan()`	exportée	plan	plan inchangé, ou refus (section 6.1)
`construire_table()`	exportée	données, rôles, plan, provenance	objet `construction` (section 6.3)

PARTIE III

Module donnees

Données, rôles et contrôles : R/donnees.R

Le module donnees reçoit la base d'observation, un data.frame, et la déclaration des rôles de ses colonnes. Il contrôle les rôles, évalue les motifs d'invalidité de chaque ligne, applique les règles d'exclusion déclarées par le plan, déduit l'unité d'observation et fusionne les intervalles d'observation d'une même tête (E2.2, E2.4, E2.7). Il ne modifie jamais les données : une ligne invalide arrête la construction, sauf règle d'exclusion déclarée, et rien n'est corrigé (E2.7). Sa fonction d'étape est preparer_donnees() ; il porte aussi le catalogue des rôles (catalogue_roles()) et la représentation commune d'une table de référence (table_reference()), toutes deux exportées. Le contenu de fond (catalogue, motifs, règles de fusion) vient de la spécification d'actuary du 16/10/2026, versée dans l'issue #11 (parties A.0 et A.1).

7.1 Conventions de temps

Trois conventions de temps sont communes à toutes les recettes (E2.3) ; elles sont appliquées par donnees et par exposition (section 8), qui ajoute l'unité de l'exposition et la fenêtre d'observation.

— **Une date est un jour** (classe Date, sans heure ni fuseau). Un intervalle d'observation est l'intervalle d'instants semi-ouvert $[\text{entrée}, \text{sortie})$: date_entree est l'instant où l'observation commence, date_sortie celui où elle cesse (jour du décès ou du rachat, ou lendemain du dernier jour couvert pour une fin de garantie). Un événement daté du jour s survient à l'instant s : la tête est exposée jusqu'à s, jour s exclu. Une ligne en_cours est ouverte à droite. *Choix de conception* : il donne des décomptes entiers et additifs, au prix d'une sous-estimation de l'exposition d'un demi-jour par événement en moyenne.

— **Âge au dernier anniversaire.** L'âge au jour t d'une tête née le jour n est le nombre d'années révolues

$$\text{age}(n,t) = \text{an}(t) - \text{an}(n) - \mathbb{1}\{(\text{mois}(t), \text{jour}(t)) < (\text{mois}(n), \text{jour}(n))\},$$

comparaison lexicographique, appliquée sans cas particulier (age_revolu()). Pour une naissance le 29 février, l'anniversaire tombe donc le 1er mars des années non bissextiles : le 28 février d'une telle année, $(2, 28) < (2, 29)$ et l'année n'est pas révolue. Une année d'âge dure ainsi 365 ou 366 jours. Convention de calcul, conséquence de la formule (spécification, A.0, T2), validée par actuary le 17/10/2026, d'impact négligeable (un jour d'exposition déplacé entre deux âges, pour les seules naissances du 29 février) ; aucune source juridique n'est établie à son sujet.

— **Période d'observation et délai de déclaration.** La période $[\text{début}, \text{fin}]$ est déclarée par le plan, fin incluse, jamais déduite des données (E2.5). Le délai de déclaration est un nombre entier de jours ≥ 0,

sans arithmétique de mois. La fin effective est fin − délai et la borne d'instant B = fin effective + 1 : l'observation s'arrête à l'instant B. Une période dont le début est postérieur à la fin effective est refusée (`periode_vide`).

7.2 Catalogue des rôles

Le moteur ne lit une colonne que par son *rôle*, déclaré par l'utilisateur dans un catalogue fermé (E2.2 ; ADR 0003, point 2). Les rôles sont transmis à `construire_table()` sous la forme d'un vecteur de chaînes nommé : noms = rôles du catalogue, valeurs = noms de colonnes de la base (ADR 0007, point 4.1). Les colonnes sans rôle sont ignorées. `catalogue_roles()`, exportée pour que l'application propose la déclaration des rôles, rend le catalogue sous la forme d'un `data.frame` de colonnes `role`, `famille`, `type_admis`, `usage` et `recettes` (tableau 7).

La colonne *usage* décrit l'emploi du rôle par la seule recette d'exposition disponible, `lignes_dates_completes` (section 8.1) :

— `requis` : rôle sans lequel aucune recette ne s'applique (refus `aucune_recette`) ;

— `facultatif` : rôle lu pour des contrôles (unicité du contrat, cohérence de l'année de naissance) ou, pour `id_tete`, pour l'unité d'observation ;

— `segmentation` : rôle requis si la segmentation du plan l'emploie (refus `role_requis_manquant`), contrôlé s'il est seulement déclaré, avec l'avertissement `role_non_employe` ;

— `non_pris_en_charge` : rôle qu'aucune recette disponible n'emploie et dont l'omission changerait le sens du résultat (pondération, agrégats, états, date de survenance distincte de la sortie, date d'inventaire, âge fourni) ; sa déclaration est **refusée** (`role_non_pris_en_charge`) plutôt qu'ignorée en silence. Le rôle `montant` est de ceux-là : la pondération par les montants attend la spécification de ses intervalles (E2.8).

Les types sont contrôlés sans conversion (E4.4) : une colonne de dates doit être de classe `Date`, exactement, à valeurs entières et finies ; une colonne `POSIXct`, une chaîne ou un nombre sont refusés (`type_incompatible`), la conversion revenant à la couche de lecture. Les modalités des rôles catégoriels sont comparées après conversion en chaîne, en respectant la casse : `H` et `F` pour `sexe` ; `deces`, `rachat`, `terme`, `autre` et `en_cours` pour `cause_sortie` (constantes `MODALITES_SEXE` et `MODALITES_CAUSE`). Un recodage (par exemple `1/2` vers `H/F`) relève de la couche de lecture.

Table 7 : Catalogue fermé des rôles (`catalogue_roles()` ; constantes `CATALOGUE_ROLES` et `TYPES_ADMIS`). La colonne `recettes` du catalogue vaut `lignes_dates_completes` pour tout rôle pris en charge, vide sinon.

Rôle	Famille	Types R admis	Usage
`id_contrat`	identification	`character`, `integer`, `double` entier fini	facultatif
`id_tete`	identification	`character`, `integer`, `double` entier fini	facultatif
`sexe`	état civil	`character`, `factor`	segmentation
`date_naissance`	état civil	`Date` (valeurs entières finies)	requis
`annee_naissance`	état civil	`integer`, `double` entier fini	facultatif
`age_fourni`	état civil	`integer`, `double` entier fini ≥ 0	non pris en charge
`date_age_fourni`	état civil	`Date` (valeurs entières finies)	non pris en charge
`date_entree`	observation	`Date` (valeurs entières finies)	requis
`date_sortie`	observation	`Date` (valeurs entières finies ; manquante admise pour une ligne `en_cours`)	requis

Rôle	Famille	Types R admis	Usage
cause_sortie	observation	character, factor	requis
date_evenement	observation	Date (valeurs entières finies)	non pris en charge
date_inventaire	observation	Date (valeurs entières finies)	non pris en charge
age	agrégats	integer, double entier fini	non pris en charge
annee	agrégats	integer, double entier fini	non pris en charge
exposition	agrégats	double fini ≥ 0	non pris en charge
evenements	agrégats	double fini ≥ 0	non pris en charge
montant	pondération	double fini > 0	non pris en charge
segment	segmentation	character, factor, integer	segmentation
etat_depart	multi-états	character, factor	non pris en charge
etat_arrivee	multi-états	character, factor	non pris en charge
date_changement	multi-états	Date (valeurs entières finies)	non pris en charge

7.3 Contrôles, refus et exclusions

Refus au niveau des rôles et des paramètres

controler_roles() contrôle la déclaration des rôles dans cet ordre, et chaque refus nomme tous les rôles et colonnes en cause : donnees_mal_formees (l'entrée n'est pas un data.frame) ; roles_mal_formes (les rôles ne sont pas un vecteur de chaînes nommé, sans valeur ni nom manquant) ; role_inconnu ; role_declare_deux_fois ; role_non_pris_en_charge ; colonne_absente ; colonne_ambigue (le nom de colonne figure plusieurs fois dans la base : lire la première sans le dire serait deviner) ; colonne_declaree_deux_fois (une colonne pour deux rôles) ; type_incompatible ; aucune_recette (un rôle requis manque ; le message nomme la recette et les rôles absents) ; role_requis_manquant (un rôle employé par la segmentation n'est pas déclaré).

controler_parametres_donnees() contrôle les champs du plan que l'étape reçoit : transition_inconnue (liste de champs autres qu'exactement depart et arrivee, ou transition non prise en charge : seules vivant $\rightarrow$ décédé et en cours $\rightarrow$ racheté le sont, constante TRANSITIONS) ; periode_invalide (liste de champs autres qu'exactement debut et fin, ou dates manquantes ou non Date) ; delai_invalide (délai qui n'est pas un entier ≥ 0) ; periode_vide ; segmentation_invalide (vecteur de chaînes sans doublon pris dans sexe et segment, vide explicite admis) ; exclusions_invalides ; exclusion_inconnue (code absent de la liste fermée des motifs). Les noms de champs sont lus exactement, jamais par appariement partiel. Le même contrôle est appelé par valider_plan(), avant toute étape : un code désigne une condition, quelle que soit la fonction qui la détecte, et le refus porte le module donnees (ADR 0007, annotation du 17/10/2026, point d).

Motifs d'invalidité des lignes

Les motifs forment une liste fermée (E2.7), évaluée par evaluer_motifs() (tableau 8).

— **Exhaustivité** : tous les motifs de ligne sont évalués sur toutes les lignes, sans court-circuit ; un motif de comparaison n'est pas levé si l'une de ses dates est manquante (le motif de la valeur manquante l'est) ;

— **motifs de tête** : évalués, quand id_tete est déclaré, sur les lignes exemptes de motif de ligne (lignes *propres*), ils marquent les lignes propres de la tête, et elles seules : une ligne déjà porteuse d'un motif de ligne n'est pas comptée de nouveau sous le motif de tête. Les lignes conservées n'en dépendent pas, puisqu'une telle ligne est de toute façon refusée ou exclue au titre de son motif de ligne ; seul le décompte du motif de tête, dans le refus et dans les diagnostics, l'omet (constat : une tête de trois lignes dont la première porte annee_naissance_incoherente et dont les naissances diffèrent donne

tete_naissance_incoherente sur les lignes 2 et 3 seulement, et le refus annonce deux lignes pour ce motif) ; le rapport est ainsi complet en un seul passage ;

— **décompte** : une ligne peut porter plusieurs motifs ; le décompte par motif compte les lignes porteuses, et le nombre de lignes distinctes est donné à part.

Table 8 : Motifs d'invalidité, dans l'ordre de restitution (constantes MOTIFS_LIGNES et MOTIFS_TETE). Ce sont aussi les codes admis comme règles d'exclusion.

Code	Condition
Motifs de ligne	
id_contrat_manquant	id_contrat déclaré et manquant
id_tete_manquant	id_tete déclaré et manquant
naissance_manquante	date de naissance manquante
entree_manquante	date d'entrée manquante
sortie_manquante	date de sortie manquante et cause autre que en_cours
cause_manquante	cause de sortie manquante
cause_hors_modalites	cause hors des cinq modalités
sexe_manquant	sexe déclaré et manquant
sexe_hors_modalites	sexe déclaré, hors de H et F
segment_manquant	segment déclaré et manquant
naissance_apres_entree	naissance postérieure à l'entrée (l'égalité est admise)
sortie_avant_entree	sortie antérieure à l'entrée (l'égalité est admise : durée nulle)
en_cours_avant_fin	cause en_cours et date de sortie renseignée antérieure à la fin déclarée : la base ne couvre pas la période déclarée
annee_naissance_incoherente	annee_naissance déclarée et différente de l'année de la date de naissance
age_aberrant	âge au dernier anniversaire au moins égal à AGE_ABERRANT (130) à la plus petite des dates de sortie et de fin de période (à la fin si la sortie manque) ; constante de catalogue, *choix de conception* : la borne laisse passer toute durée de vie humaine connue, elle n'est pas un paramètre du plan
contrat_duplique	id_contrat déclaré et présent sur plusieurs lignes (toutes les lignes du groupe sont listées)
Motifs de tête (si id_tete est déclaré)	
tete_naissance_incoherente	dates de naissance différentes entre lignes d'une même tête
tete_sexe_incoherent	sexes différents au sein d'une tête (sexe déclaré)
tete_segment_incoherent	segments différents au sein d'une tête, si la segmentation emploie segment
tete_observee_apres_deces	le premier décès s_d de la tête est antérieur à B et une ligne de la tête a une fin d'observation postérieure à s_d (lecture B de E2.4, ci-dessous)

Refus par défaut et exclusions déclarées

Une ligne qui porte un motif **non déclaré** dans les règles d'exclusion du plan arrête la construction (refus ligne_invalide), même si elle porte aussi un motif déclaré. Le message nomme, pour chaque motif en cause, le nombre de lignes et leurs dix premiers numéros (NB_LIGNES_CITEES, numéros 1 à n des lignes du data.frame reçu), puis le total des lignes distinctes ; la condition porte la liste complète (E2.7, E5.5).

Le plan peut déclarer des *règles d'exclusion*, paramètre exclusions de l'étape donnees : un vecteur de codes de motif sans doublon, vide explicite admis. Une ligne est exclue si l'un de ses motifs est déclaré. Les exclusions sont comptées et listées par motif dans le résultat de l'étape (resultat$exclusions,

`resultat$lignes_exclues`) et chacune donne un avertissement `exclusion`. Si toutes les lignes sont exclues, la construction est refusée (`aucune_ligne_conservee`) ; une base sans ligne l'est aussi (`base_vide`). Le diagnostic `diagnostics$motifs` restitue, pour chaque motif, sa nature (ligne ou tête), son décompte et s'il est exclu.

Les lignes conservées sont celles de l'entrée : `donnees` lui-même s'il n'y a aucune exclusion, sinon le sous-ensemble de ses lignes, sans aucune modification de valeur ; les quantités dérivées (observations fusionnées) sont rangées à part. Une réserve technique : l'extraction des lignes conservées par l'opérateur d'indexation de R retire des colonnes les attributs autres que la classe et les niveaux d'un facteur (par exemple `tzone`, `label` ; constat sur R 4.3.3) ; les valeurs et les classes ne changent pas, et le moteur ne lit pas ces attributs.

7.4 Unité d'observation et fusion par tête

Unité d'observation, déduite des rôles

L'unité d'observation n'est pas un paramètre du plan : le moteur la déduit des rôles et de la transition, comme il déduit la recette (décision M48 ; ADR 0007, annotation du 17/10/2026, point a), et l'inscrit dans les paramètres effectifs de l'étape et dans la fiche :

— **tête**, si l'état de départ de la transition est un état de la personne (vivant → décédé) et que `id_tete` est déclaré ;

— **contrat** sinon : transition dont l'état de départ est un état du contrat (en cours → racheté, décision M46), ou transition de tête sans `id_tete`. Dans ce second cas, l'avertissement `unite_contrat` est inscrit dans l'objet et repris dans la fiche : « une tête titulaire de plusieurs contrats est comptée plusieurs fois, ce qui surestime l'exposition et peut compter plusieurs fois un même décès » (cas F4 des tests : deux décès comptés au lieu d'un).

Un paramètre d'unité serait soit redondant avec les rôles, soit en contradiction avec eux : une tête déclarée « contrat » alors que `id_tete` est présent compterait plusieurs fois un même décès. Pour le rachat, l'unité est le contrat ; l'identifiant de tête sert alors aux contrôles de cohérence et au futur rééchantillonnage par têtes (E2.4, E2.14) : il est gardé dans la table des observations.

Fusion des intervalles d'une tête

En unité tête, les intervalles $[\text{entrée}, \text{fin})$ des lignes d'une même tête (fin $= +\infty$ pour une ligne `en_cours`) sont triés par entrée et fusionnés quand entrée suivante $\leq$ fin courante, ce qui couvre le chevauchement et la contiguïté exacte (`fusionner_intervalles()`). **Aucune tolérance « à un jour près »** : un écart d'un jour reste un trou non exposé, et la tête est de nouveau tronquée à gauche à sa réentrée ; combler l'écart inventerait une exposition. Un tel écart trahit souvent une date de fin inclusive dans la base : il est relevé et décompté à part, non corrigé (*choix de conception*). La cause de fin d'une composante fusionnée est `deces` si une ligne de décès se termine à cette fin, `en_cours` si la fin est $+\infty$, `censure` sinon ; le décès n'est compté qu'une fois.

Le calcul de la fusion est vectorisé par un maximum cumulé décalé par tête, exact en `double` tant que (nombre de têtes) $\times$ (étendue des dates, en jours, $+1$) reste inférieur à 2^{52}. Au-delà, le moteur refuse (`dates_hors_plage`) au lieu de fusionner inexactement ; pour 10^6 têtes, il faudrait une étendue d'environ $4{,}5 \cdot 10^9$ jours, ce que seules des dates aberrantes atteignent. C'est une limite d'implémentation, non un motif de ligne : elle n'est pas excluable.

Observation après le décès : lecture B de E2.4

Une tête dont un contrat est observé après son décès a une trajectoire indéterminée, et ses lignes sont refusées (E2.4). Le décès doit cependant être connu : le mainteneur a retenu une *lecture B* (décision M48) : le motif `tete_observee_apres_deces` n'est levé que si le premier décès s_d de la tête est **antérieur à la borne** B de la période effective et qu'une fin d'observation d'une ligne de la tête le dépasse. Une ligne `en_cours` compte pour $+\infty$, quelle que soit sa date de sortie, et une sortie manquante aussi (`deces_par_tete()`). Deux lignes closes par le décès le même jour sont cohérentes.

Si $s_d \geq B$, le décès est hors de la période effective : aucune ligne n'est refusée, mais le relevé `observations_apres_deces_hors_periode` compte ces têtes et donne un avertissement du même code. Ce relevé est calculé **sur les seules lignes conservées** : une tête écartée par une exclusion déclarée n'y est pas comptée, une tête dont une ligne seulement est exclue l'est sur ses lignes conservées. Il est calculé dès que `id_tete` est déclaré, **quelle que soit l'unité**, donc aussi en unité contrat pour le rachat (contrôle par identifiant de tête, E2.4, M46, M48) ; sans `id_tete`, il n'est pas calculable et vaut `NA`.

Relevés et avertissements

L'étape restitue dans `diagnostics$releves`, sans bloquer, le nombre de têtes et de têtes multi-contrats, de chevauchements, de contiguïtés, d'intervalles disjoints, d'écarts d'un jour, de fenêtres vides (observations sans intersection avec la période), d'observations, et le relevé après décès hors période. Les relevés de fusion valent `NA` sans `id_tete`. Ses avertissements sont : `unite_contrat` ; `role_non_employe` ; `exclusion` ; `chevauchements`, `disjoints` et `ecarts_un_jour`, en unité tête seulement ; `fenetres_vides` et `observations_apres_deces_hors_periode`, dans toutes les unités. Les contiguïtés sont décomptées sans avertissement : une contiguïté exacte est fusionnée sans perte.

Le résultat de l'étape (`resultat`) contient les lignes conservées et leurs numéros, les exclusions, l'unité retenue, la table des observations (une ligne par composante fusionnée ou par contrat : identifiant, numéro de la première ligne, nombre de lignes, date de naissance, sexe s'il est déclaré, segment s'il est employé, début, fin, cause) et, pour chaque ligne conservée, le numéro de son observation. Les paramètres effectifs (`parametres`) sont le code interne de la transition (`vivant_decede` ou `en_cours_rachete`), l'unité d'observation, la segmentation, les exclusions et la fin effective.

7.5 Table de référence

Toute table de référence — loi vraie synthétique du simulateur, table embarquée, table importée — est lue par le moteur sous une seule forme : la classe S3 `table_reference`, sur un `data.frame` en format long (ADR 0007, point 7.1, et annotation du 17/10/2026, point b), que construit et contrôle `table_reference(x, nom, statut, source, parametres, forme_origine)`, exportée pour l'import de tables par l'application :

— colonnes `age` (entier ≥ 0), `sexe` (`H`, `F`, ou `ensemble` pour une table sans distinction de sexe, modalité qui ne coexiste jamais avec `H` ou `F`, dans la table ou au sein d'une même année), `qx` (double fini dans $[0, 1]$) et, facultative, `annee` (entier), pour les tables prospectives de la v2 ;

— âges sans doublon, croissants dans l'ordre reçu et contigus, au sein de chaque sexe (et de chaque année) ;

— attributs `nom`, `statut` (`synthetique`, `embarquee` ou `importee`), `source` (arrêté, URL, fichier de provenance), `forme_origine` (`qx` ou `lx`, défaut `qx`) et `parametres` : une liste non vide est **requise** pour une table synthétique, qui n'a d'autre provenance que sa loi, et **interdite** pour une table embarquée ou importée, qu'elle ferait passer pour dérivée d'une loi.

Le constructeur refuse au lieu de corriger, avec un code par contrôle : table_nom_manquant, table_statut_inconnu, table_source_manquante, table_parametres_manquants, table_parametres_inattendus, table_forme_inconnue, table_mal_formee, table_colonne_manquante, table_colonne_inconnue, table_vide, table_colonne_mal_typee, table_sexe_hors_modalites, table_sexes_melanges, table_qx_non_fini, table_qx_hors_bornes, table_age_double, table_ages_non_croissants et table_ages_non_contigus.

7.6 Vérifications

tests/unitaires/test_donnees.R vérifie le catalogue, chaque refus de rôle et de paramètre, chaque motif et les règles d'évaluation, les exclusions (lignes conservées identiques à l'entrée), la lecture B de E2.4 au jour près de la borne B (délais 0 et 30 jours) et le relevé après décès hors période (lignes conservées, deux unités), la table de référence et ses refus. Les cas de fusion F1 à F7 de la spécification (chevauchement, contiguïté, écart d'un jour, décès sur deux contrats, observation après décès, deux décès, naissances discordantes) ont des attendus calculés à la main ; la fusion est vérifiée en outre contre un second algorithme, une boucle naïve écrite dans le test.

7.7 Fonctions du module

Table 9 : Définitions de premier niveau de R/donnees.R.

Nom	Statut	Entrées	Sortie et rôle
CATALOGUE_ROLES	constante	—	catalogue fermé des rôles (rôle, famille, code de type, usage)
TYPES_ADMIS	constante	—	libellé des types R admis par code de type
RECETTE_BRANCHE	constante	—	seule recette d'exposition disponible : lignes_dates_completes
MODALITES_SEXE	constante	—	H, F
MODALITES_CAUSE	constante	—	deces, rachat, terme, autre, en_cours
ROLES_SEGMENTATION	constante	—	sexe, segment
AGE_ABERRANT	constante	—	130 (motif age_aberrant)
TRANSITIONS	constante	—	transitions prises en charge : départ, arrivée, code interne, porteur de l'état de départ (tête ou contrat)
MOTIFS_LIGNES	constante	—	motifs de ligne (tableau 8)
MOTIFS_TETE	constante	—	motifs de tête (tableau 8)
NB_LIGNES_CITEES	constante	—	10 numéros de ligne cités par motif dans un message de refus
catalogue_roles()	exportée	—	catalogue des rôles (tableau 7)
composantes_date()	interne	jours depuis 1970-01-01	année, mois, jour
age_revolu()	interne	naissance, date	âge au dernier anniversaire (section 7.1)
entiers_finis()	interne	vecteur	valeurs non manquantes entières et finies
type_conforme()	interne	colonne, code de type	conformité au type du catalogue, sans conversion
liste_guillemets()	interne	chaînes	liste entre guillemets pour les messages
message_aucune_recette()	interne	rôles requis, manquants	message du refus aucune_recette, partagé avec exposition

Nom	Statut	Entrées	Sortie et rôle
controler_roles()	interne	données, rôles, segmentation	refus au niveau des rôles ; rend les rôles inchangés
controler_parametres_donnees()	interne	transition, période, délai, segmentation, exclusions	refus des paramètres ; transition retenue, début, fin, borne B
groupes_heterogenes()	interne	groupes, valeurs	groupes où la valeur n'est pas constante
extremes_par_groupe()	interne	groupes, valeurs	minimum et maximum par groupe
deces_par_tete()	interne	têtes, sorties, causes	premier décès de chaque tête et observation au-delà
evaluer_motifs()	interne	données, rôles, période, borne, segmentation	numéros des lignes porteuses de chaque motif
fusionner_intervalles()	interne	têtes, entrées, fins	composantes fusionnées et relevés ; refus dates_hors_plage
preparer_donnees()	interne	données, rôles, transition, période, délai, segmentation, exclusions	fonction d'étape donnees : paramètres effectifs, résultat, diagnostics, avertissements
STATUTS_TABLE	constante	—	synthetique, embarquee, importee
SEXES_TABLE	constante	—	H, F, ensemble
FORMES_ORIGINE	constante	—	qx, lx
table_reference()	exportée	table, nom, statut, source, paramètres, forme d'origine	objet table_reference contrôlé (section 7.5)

Partie IV

Module exposition

Exposition et événements : R/exposition.R

Le module exposition calcule, pour la transition étudiée, l'exposition au risque et le nombre d'événements par cellule (segment, âge et, sur demande, année civile), en traitant explicitement la troncature à gauche et la censure à droite (E2.3, E2.5, E2.6 ; ADR 0003, points 3, 6 et 8). Sa fonction d'étape est calculer_exposition() ; elle reçoit la table des observations de l'étape donnees (composantes fusionnées ou contrats, section 7.4), les rôles, les champs du plan et ses deux paramètres. Le contenu de fond vient de la spécification d'actuary (issue #11, parties A.0 et A.3) et de son erratum du 17/10/2026.

8.1 Recette d'exposition et conventions d'âge

Recette déduite des rôles

La *recette d'exposition* n'est pas un paramètre du plan : le moteur retient la plus fine que les rôles permettent, la nomme dans les paramètres effectifs et dans la fiche, et **refuse si aucune ne s'applique**, sans jamais deviner (E2.3 ; ADR 0003, point 3 ; ADR 0007, point 5.3). Une seule recette existe dans cette branche (tableau 10) ; les recettes « agrégats » et « stocks successifs » (E2.1) viendront avec les branches qui les créent. deduire_recette() retient la recette dont tous les rôles requis sont déclarés, ou refuse (aucune_recette, le message nommant la recette et les rôles absents) ; dans construire_table(), le même manque est détecté plus tôt par le contrôle des rôles, avec le même code et le module donnees.

Table 10 : Recettes d'exposition (constantes RECETTE_BRANCHE et CONVENTION_NATIVE).

Recette	Rôles requis	Convention native
lignes_dates_completes	date_naissance, date_entree, date_sortie, cause_sortie (lignes individuelles, dates complètes)	âge au dernier anniversaire (dernier_anniversaire)

Conventions d'âge

Les conventions d'âge sont ordonnées de la plus fine à la plus grossière (ORDRE_CONVENTIONS) : âge au dernier anniversaire (dernier_anniversaire), âge atteint dans l'année (age_atteint_annee), âge fourni (age_fourni). Le paramètre convention de l'étape vaut plus_fine, qui retient la convention native de la recette, ou désigne une convention :

— une convention **plus grossière** que la native est un *forçage*, admis : il est inscrit dans les paramètres effectifs et dans la fiche (convention demandée, native, retenue, forcée ou non) et donne l'avertissement convention_forcee (E2.3) ;

— une convention **plus fine** que la native est refusée (`convention_plus_fine`) : les rôles ne la permettent pas ;

— `age_fourni`, non mise en œuvre (ses rôles ne sont pas pris en charge), est refusée (`convention_non_mise_en_oeuvre`) ; toute autre valeur aussi (`convention_invalide`) ; `par_annee` doit valoir `TRUE` ou `FALSE` (`par_annee_invalide`).

Chaque convention définit ses cellules d'âge et l'unité de l'exposition (`UNITES_EXPOSITION`, inscrite dans la fiche) :

— `dernier_anniversaire` : la cellule d'âge x d'une tête est $[\text{anniv}(x), \text{anniv}(x+1))$, l'âge étant celui de la section 7.1 (29 février compris) ; l'unité est l'**année d'âge** : un morceau de j jours dans la cellule x vaut j/L_x, avec $L_x = \text{anniv}(x+1) - \text{anniv}(x)$, soit 365 ou 366 jours ;

— `age_atteint_annee` : l'âge au jour t est $\text{an}(t) - \text{an}(n)$, la cellule est l'année civile [1er janvier, 1er janvier suivant), et l'unité la fraction d'année civile ($j/365$ ou $j/366$).

Pourquoi l'année d'âge. Avec cette unité, une année d'âge entière vaut exactement 1 ; c'est la seule pour laquelle l'hypothèse « force constante par âge entier » donne exactement $q_x = 1 - e^{-\mu_x}$ pour toute année d'âge (résultat exact). Avec une division par 365,25, une année d'âge pèserait $365/365{,}25$ ou $366/365{,}25$ selon les années, et la correspondance ne serait plus exacte. L'unité est un *choix de conception* de la spécification (A.0, T3), validé par `actuary` le 17/10/2026 (validation de fin de branche).

Statut des valeurs. L'exposition est produite en jours, entiers exacts stockés en `double` : l'exposition totale en jours d'un portefeuille de 10^6 lignes sur 30 ans (environ $1{,}1 \cdot 10^{10}$) dépasse `.Machine$integer.max` ; résultat exact. L'exposition en unité d'exposition est une somme de quotients en virgule flottante : approximation numérique, vérifiée à 10^{-12} près en relatif. L'arithmétique des dates du module (années bissextiles, anniversaires, 1er janvier) est entière et vectorisée, écrite indépendamment de celle du simulateur ; elle est recoupée par `as.Date()` et comparée à `age_revolu()` du module `donnees`, écrite indépendamment, dans les tests.

8.2 Troncature à gauche et censure à droite

Fenêtre d'observation et décompte

Pour une observation d'entrée e et de fin s ($s = +\infty$ si elle est en cours), la période effective étant $[D, B)$, avec D le début de la période et B la borne de la section 7.1 :

— la **fenêtre** est $[a, b)$, avec $a = \max(e, D)$ et $b = \min(s, B)$, vide si $a \geq b$: une entrée antérieure à D est tronquée à gauche à D, et l'observation est censurée à droite à B (fin de période et délai de déclaration) ;

— l'**exposition centrale** de la cellule (g, x) est

$$E^c_{g,x} = \sum_{i \in g} \big| [a_i, b_i) \cap [\text{anniv}_i(x), \text{anniv}_i(x+1)) \big|,$$

en jours, et en unité d'exposition par division de chaque morceau par la longueur de sa cellule ;

— l'**événement** de la transition étudiée est compté si et seulement si la cause est la cible et $D \leq s < B$; il est attribué à la cellule du jour s. Un décès le jour d'un anniversaire est donc attribué à l'âge $x+1$, où l'exposition de la tête est nulle (cas H8 des tests), et un événement le jour de l'entrée est compté avec une exposition nulle (cas H11) ; le relevé `evenement_sans_exposition` compte les cellules à exposition nulle qui comptent un événement.

Sorties concurrentes

Une sortie dans la fenêtre dont la cause n'est ni la cible ni `en_cours` est une *sortie concurrente* : rachat ou décès pour l'autre transition, terme, autre cause, composante de tête close sans décès. Elle est traitée comme une **censure à droite à son instant**, et comptée dans la colonne `sorties_concurrentes` de sa cellule, censures programmées comprises. Ce traitement suppose une **censure non informative** (risques concurrents indépendants) : l'hypothèse est nommée dans les paramètres effectifs et dans la fiche (`HYPOTHESE_CENSURE` ; E2.6).

Compléments de l'exposition initiale

L'exposition initiale n'est pas choisie dans cette branche : pour chaque événement compté au jour s dans une cellule qui se termine en $\mathrm{fin}(x)$, deux compléments, en unité d'exposition, sont restitués :

— `complement_initial_cellule` : $c_1 = \mathrm{fin}(x) - s$, jusqu'à la fin de la cellule (anniversaire suivant sous `dernier_anniversaire`, 1er janvier suivant sous `age_atteint_annee`), convention actuarielle classique ;

— `complement_initial_fenetre` : $c_2 = \min(\mathrm{fin}(x), B) - s$, jusqu'à la fin de la fenêtre, la censure par la période étant connue d'avance.

D'où deux expositions initiales, $E^{i,\text{cellule}} = E^c + \sum c_1$ et $E^{i,\text{fenêtre}} = E^c + \sum c_2$. Aucune n'est sans biais indépendamment de l'hypothèse de répartition dans l'année. *Résultat exact* : pour une cohorte entrée à l'âge exact x, sans sortie concurrente, censurée par la période à un instant $c \sim \mathcal{U}(0,1)$ indépendant de la durée de vie, la limite presque sûre de $D/E^{i,\text{fenêtre}}$ quand le nombre de têtes croît (rapport des espérances) vaut $2\big(1-(1-e^{-\mu})/\mu\big)$ sous force constante, soit environ $0{,}05043$ pour $q = 0{,}05$, et exactement q sous l'hypothèse de répartition uniforme ; dans le même schéma, la limite de $D/E^{i,\text{cellule}}$ vaut exactement q sous l'hypothèse de Balducci (calcul direct). À effectif fini, le rapport a un biais que ce résultat ne couvre pas. Le choix de l'exposition initiale revient donc à la spécification de l'estimateur binomial (branche des taux bruts), qui devra nommer son hypothèse de répartition. Dans les deux définitions, une sortie concurrente reste à son instant réel : c'est une limite connue de l'estimateur binomial, à documenter avec lui.

Table d'exposition

`resultat$exposition` a une ligne par cellule : clés de segmentation, `age` et, avec `par_annee`, `annee` ; colonnes `exposition_jours`, `exposition_centrale` (unité nommée dans `parametres$unite_exposition`), `complement_initial_cellule` et `complement_initial_fenetre` (sans `par_annee` seulement), `evenements` et `sorties_concurrentes`. Par segment, la table couvre la plage **contiguë** des âges dont les cellules ont une exposition non nulle, un événement ou une sortie concurrente ; les zéros sont explicites. Inclure les sorties concurrentes dans la plage garde exacte l'identité « somme de `sorties_concurrentes` = nombre de sorties concurrentes dans la fenêtre » ; la plage de validité de la table finale se fixera à la fermeture (erratum A.3, point 1).

Clés de segmentation. Leur ordre et leur type ne dépendent ni des niveaux d'un facteur ni de la classe reçue : `sexe`, et `segment` reçu en chaîne ou en facteur, sont rendus en chaînes et triés en ordre d'octets ; un `segment` entier est rendu en entier et trié par valeur. L'empreinte des données ignore les niveaux d'un facteur (section 15.2) : sans cette règle, deux entrées de même empreinte pourraient donner des tables ordonnées différemment (erratum A.3, point 4).

Paramètres effectifs, diagnostics, avertissements. Les paramètres effectifs sont la recette, la convention (demandée, native, retenue, forcée), l'unité d'exposition, `par_annee`, le code de la transition, la

segmentation, la période, le délai, la fin effective et l'hypothèse de censure. Les diagnostics donnent le nombre d'observations et d'observations exposées, l'exposition totale en jours, le nombre d'événements et de sorties concurrentes, le nombre de cellules et de cellules à exposition nulle qui comptent un événement. Avertissements : `convention_forcee`, `evenement_sans_exposition` et `aucune_exposition` (aucune exposition ni aucun événement dans la période).

8.3 Découpage par année

Avec `par_annee = TRUE`, chaque morceau d'exposition est en outre découpé au 1er janvier, et un événement est attribué à l'année civile de son jour. La grille est **rectangulaire** : elle croise la plage d'âges du segment et toutes les années civiles de l'année du début de la période à celle de la fin effective, avec des zéros explicites ; la première et la dernière année peuvent être partielles (erratum A.3, point 2). La somme sur les années redonne la table par âge seul, exactement en jours. Les compléments de l'exposition initiale ne sont pas produits dans ce découpage.

8.4 Vérifications

Les cas construits à la main de la spécification (H1 à H13 pour une ligne, F1 à F4 pour la fusion) sont vérifiés par `tests/unitaires/test_exposition.R`, en jours exactement et en unité d'exposition à 10^{-12} près. Quelques-uns illustrent les règles ci-dessus (période du 2015-01-01 au 2024-12-31, délai nul, vivant $\rightarrow$ décédé) :

Table 11 : Exemples de cas calculés à la main (spécification, A.4 ; valeurs reproduites par le moteur sur R 4.3.3).

Cas	Ligne	Exposition par âge (jours / L_x)	Événement
H2	née 1945-10-15 ; 2016-05-20 $\rightarrow$ 2019-02-10, décès	70 : 148/366 ; 71 : 365/365 ; 72 : 365/365 ; 73 : 118/365 ; total 996 jours	à 73 ans ; $c_1 = c_2 = 247/365$
H6	née 1964-02-29 ; 2021-01-01 $\rightarrow$ en cours	56 : 59/366 ; 57 à 59 : 1 chacun ; 60 : 307/366 ; total 1 461 jours, 4 années d'âge	aucun
H8	née 1955-04-12 ; 2019-09-01 $\rightarrow$ 2020-04-12, décès	64 : 224/366 ; 65 : 0	à 65 ans, exposition nulle à 65 ; $c_1 = 365/365$
H11	née 1980-01-01 ; 2018-06-15 $\rightarrow$ 2018-06-15, décès	aucune	à 38 ans ; $c_1 = 200/365$
H12	H2 avec `age_atteint_annee`	71 : 226/366 ; 72 : 1 ; 73 : 1 ; 74 : 40/365	à 74 ans ; $c_1 = 325/365$

Le cas F4 (décès porté par deux contrats d'une même tête) donne un décès en unité tête et deux en unité contrat, avec l'avertissement `unite_contrat`. Sur un portefeuille simulé, `tests/unitaires/test_construire_table.R` vérifie des *identités exactes* : exposition totale en jours égale à la somme des $\max(0, b - a)$, égalité entière ; décès comptés égaux aux décès simulés tels que $D \leq s \leq$ fin effective ; même identité pour les rachats, décès censurés ; décès par sexe et par âge égaux aux décès simulés par âge au dernier anniversaire ; avec `par_annee`, somme sur les années égale à l'exposition par âge seul, en jours. La segmentation est vérifiée avec des facteurs aux niveaux permutés, en chaîne et en entier.

8.5 Fonctions du module

Table 12 : Définitions de premier niveau de R/exposition.R.

Nom	Statut	Entrées	Sortie et rôle
ORDRE_CONVENTIONS	constante	—	conventions d'âge, de la plus fine à la plus grossière
CONVENTIONS_MISES_EN_OEUVRE	constante	—	dernier_anniversaire, age_atteint_annee
VALEURS_CONVENTION	constante	—	valeurs admises du paramètre convention
CONVENTION_NATIVE	constante	—	convention native de chaque recette
UNITES_EXPOSITION	constante	—	unité de l'exposition par convention
HYPOTHESE_CENSURE	constante	—	hypothèse de censure non informative, inscrite dans la fiche
deduire_recette()	interne	rôles	recette, convention native, rôles requis ; refus aucune_recette
controler_parametres_exposition()	interne	convention, par_annee	refus des valeurs
resoudre_convention()	interne	convention demandée, native	convention retenue et forçage ; refus convention_plus_fine
an_bissextile()	interne	années	bissextilité grégorienne
bissextiles_anterieures()	interne	années	nombre d'années bissextiles antérieures
premier_janvier()	interne	années	jour du 1er janvier
JOURS_AVANT_MOIS	constante	—	jours écoulés avant chaque mois, année commune
annee_du_jour()	interne	jours	année civile
composantes_jour()	interne	jours	année, mois, jour, quantième commun
calendrier()	interne	années extrêmes	table calendaire
indice_annee()	interne	table, années	indice dans la table calendaire
annee_calendrier()	interne	table, jours	année civile, par la table
anniversaire_age()	interne	table, naissances, âges	jour de l'anniversaire
age_dernier_anniversaire()	interne	table, naissances, jours	âge au dernier anniversaire
debut_cellule()	interne	table, naissances, âges, convention	début de la cellule d'âge
cellule_du_jour()	interne	table, naissances, jours, convention	cellule d'âge du jour
extraire_composantes()	interne	composantes, indices	sous-ensemble
decouper_cellules()	interne	fenêtres, convention	morceaux contenus chacun dans une cellule
decouper_annees()	interne	morceaux	morceaux découpés au 1er janvier
somme_par_cellule()	interne	cellules, valeurs	sommes par cellule, zéros explicites
calculer_exposition()	interne	observations, rôles, transition, période, délai, segmentation, convention, par_annee	fonction d'étape exposition

PARTIE V

Module brut

Taux bruts : R/brut.R

Module prévu (E2.12, E2.14) : estimateurs des taux bruts et leurs intervalles de confiance exacts et asymptotiques. Il sera créé par la branche des taux bruts (docs/feuille-de-route.md, § 2).

9.1 Fonctions du module

TABLE 13 : Définitions de premier niveau de R/brut.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/brut.R absent			

PARTIE VI

Module lissage

Lissage : R/lissage.R

Module prévu (E2.12) : lissages des taux bruts. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

10.1 Fonctions du module

TABLE 14 : Définitions de premier niveau de R/lissage.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/lissage.R absent			

PARTIE VII

Module positionnement

Positionnement sur une table de référence : R/positionnement.R

Module prévu (E2.12, E2.15) : positionnement d'une table d'expérience sur une table de référence. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

11.1 Fonctions du module

TABLE 15 : Définitions de premier niveau de R/positionnement.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/positionnement.R absent			

PARTIE VIII

Module prospective

Tables prospectives : R/prospective.R

Module prévu (E2.13, v2) : tables prospectives, toute table prospective étant une surface de mortalité.

12.1 Fonctions du module

TABLE 16 : Définitions de premier niveau de R/prospective.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/prospective.R absent			

PARTIE IX

Module fermeture

Fermeture aux âges extrêmes : R/fermeture.R

Module prévu (E2.12) : fermeture de la table aux âges extrêmes. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

13.1 Fonctions du module

TABLE 17 : Définitions de premier niveau de R/fermeture.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/fermeture.R absent			

PARTIE X

Module validation

Validation et suivi d'expérience : R/validation.R

Module prévu (E2.12, E2.18) : validation de la table construite et suivi d'expérience. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

14.1 Fonctions du module

TABLE 18 : Définitions de premier niveau de R/validation.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/validation.R absent			

Partie XI

Module rapport

Fiche de construction et empreintes : R/rapport.R

Le module rapport porte les quantités des rapports, pas leur rendu (E4.1) : dans cette branche, la fiche de construction en mémoire et l'empreinte des données reçues. Le rapport figé HTML, l'écriture des tables et la sérialisation de la fiche viendront avec la branche claude/fiche-rapport (ADR 0007, point 12).

15.1 Fiche de construction

La *fiche de construction* est la description lisible par machine d'une construction, qui suffit à la rejouer à l'identique (E4.5 ; ADR 0005, point 6 ; ADR 0007, point 6.2). Elle est assemblée par assembler_fiche() et rangée dans le champ fiche de l'objet. Elle ne contient **aucun horodatage** : deux constructions identiques ont des fiches identiques. Ses champs :

— version_fiche (VERSION_FICHE, 1) ; versions (tablexp, lue dans la description du package installé, et R.version.string) ; plateforme (système, version, machine, BLAS par extSoftVersion(), LAPACK par La_library()) ;

— roles ; plan, le plan **reçu, inchangé** ;

— transition et son code interne transition_code ; periode, delai_declaration et fin_effective ; segmentation ;

— unite_observation et, s'il y a lieu, avertissement_unite (message de l'avertissement unite_contrat) ;

— recette ; convention (demandée, native, retenue, forcée) ; unite_exposition ; par_annee ; exclusions (par motif, avec leur nombre) ; hypothese_censure ;

— graine et graine_mention (NA et « aucune étape aléatoire » dans cette branche) ; generateur (GENERATEUR) ;

— provenance et provenance_mention (« aucun fichier (données construites en mémoire) » si provenance est NULL) ;

— empreintes (section 15.2) ; avertissements.

Les champs qui n'ont pas d'objet (convention et unité d'exposition d'un plan sans étape exposition) valent NULL ou NA. **Rejeu** : construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance) redonne un objet identical() hors execution ; c'est un test de la batterie de reproductibilité. La fiche vit en mémoire : son format d'écriture et de relecture est reporté, sous des contraintes déjà posées (texte

lisible par un relecteur, aller-retour identique, nombres compris, aucune évaluation de code à l'import, R de base, version de fiche ; ADR 0007, point 12.2).

15.2 Empreinte des données

Algorithme et portée

L'empreinte identifie l'entrée reçue par le moteur ; elle n'a pas de rôle de sécurité. Elle est un MD5, calculé par `tools::md5sum()` sur un fichier temporaire écrit en binaire (`empreinte_donnees()`), l'algorithme étant nommé dans la fiche (`empreintes$algorithme`, `"md5"`) pour qu'un changement ultérieur soit visible et daté (décision M45, P8 ; ADR 0007, point 9). R 4.3.3 n'offre ni SHA-256 ni empreinte directe d'un vecteur d'octets dans `tools` (ADR 0007, mesures), et le moteur n'admet aucun package hors priorité `base`. L'empreinte n'est pas calculée sur `serialize()` : l'en-tête de la sérialisation porte la version de R qui écrit, si bien qu'à données identiques l'empreinte changerait avec la version de R. Elle est calculée sur une *représentation textuelle canonique*, définie indépendamment de la manière de l'écrire, qu'un tiers peut recalculer avec n'importe quel outil MD5.

La fiche porte : `algorithme` ; `representation` (`ENTETE_CANONIQUE`, `"tablexp canonique 1"`) ; `donnees` ; `fichier` et `algorithme_fichier`, reçus par `provenance` (`NA` sans fichier) ; `resultat` et `code`, à `NA` avec le statut « non calculée (branche fiche-rapport) » (`statut_resultat_code`) : l'empreinte du résultat dépend de la plateforme (E3.6) et attend la sérialisation, et celle du code suppose de fixer ce qu'est « le code ». L'argument `provenance` est `NULL` ou une liste de trois chaînes non vides, `fichier`, `empreinte` et `algorithme` ; sinon, refus `provenance_invalide` (`controler_provenance()`).

Représentation canonique

`texte_canonique()` écrit les octets suivants (ADR 0007, point 9.3, et annotation du 17/10/2026, point f) :

— **périmètre** : les seules colonnes auxquelles un rôle est déclaré, **toutes les lignes reçues**, avant exclusions, dans l'ordre reçu ; une base dont les lignes sont permutées a une autre empreinte, à résultats égaux. Les exclusions sont des paramètres du plan, non une propriété de l'entrée ;

— **ordre des colonnes** : par nom de rôle, en ordre d'octets ; le nom de colonne n'entre pas dans le texte (il est dans `fiche$roles`) : renommer une colonne ne change pas l'empreinte ;

— **structure** : une première ligne `tablexp canonique 1<TAB><nombre de lignes><TAB><nombre de colonnes>`, puis, pour chaque colonne, une ligne d'en-tête `#<rôle><TAB><type>` suivie d'une valeur par ligne ; lignes terminées par LF, aucun CR, fichier écrit octet par octet ;

— **valeurs** : `double` écrit en `%.17g` après ajout de 0 (le zéro négatif est normalisé ; `%.17g` restitue un double au bit près, `%.15g` ne le fait pas), `NA`, `NaN`, `Inf`, `-Inf` ; `integer` en `%d` ; `logical` en `TRUE`, `FALSE`, `NA` ; `Date` en `%Y-%m-%d` ; chaînes et facteurs (convertis en chaînes, les niveaux n'entrent pas) : octets UTF-8 entre guillemets droits, avec cinq échappements (`\\`, `\n`, `\r`, `\t`, `\"`), `NA` sans guillemets pour une valeur manquante ;

— **codage** : les chaînes sont converties en UTF-8 puis traitées en octets ; le moteur refuse une chaîne au codage `bytes` (`chaine_codage_bytes`), une chaîne qui n'est pas de l'UTF-8 valide (`chaine_non_utf8`) et, dans une session dont la locale n'est ni UTF-8 ni latin-1, une chaîne non ASCII non marquée, dont le codage est indéterminé (`chaine_codage_indetermine`). Sous ces règles, l'empreinte ne dépend pas des paramètres régionaux.

Le libellé de type de l'en-tête est pris dans une **liste fermée de six chaînes**, par des tests faits dans l'ordre du tableau 19 (`colonne_canonique()`) ; le premier qui réussit fixe le libellé. Un type hors de la liste est

refusé (type_non_canonique), sans conversion. C'est une seconde barrière : le contrôle des types des rôles (type_incompatible, section 7.2) refuse déjà, avant le calcul de l'empreinte, toute classe hors du catalogue ; aucun rôle n'admet aujourd'hui une colonne logique, si bien que le libellé logical n'est pas produit par construire_table().

Table 19 : Libellés de type de la représentation canonique, dans l'ordre des tests.

Ordre	Libellé	Test
1	factor	facteur, ordonné ou non
2	Date	classe exactement Date (une classe Date augmentée échoue et tombe au test 3)
3	refus	toute autre classe (POSIXct, difftime, AsIs…)
4	character	chaîne
5	integer	entier
6	double	double
7	logical	logique
8	refus	tout autre type de base (liste, complexe, octets bruts…)

Pourquoi factor reste distinct de character. Les valeurs s'écrivent de la même façon ; seul le libellé distingue une colonne facteur d'une colonne de chaînes aux mêmes valeurs. L'empreinte identifiant l'entrée, deux data.frame dont une colonne n'a pas la même classe sont deux entrées différentes, que la fiche doit permettre de distinguer. L'empreinte ne dépend pas, en revanche, des niveaux d'un facteur ; c'est pourquoi aucun résultat ne doit en dépendre (clés de segmentation, section 8.2).

Règle durable. Changer la représentation canonique (libellé, ordre des tests, format d'une valeur) change l'empreinte des données de toutes les fiches : c'est un changement de résultat, qui appelle l'incrément de version_fiche et le visa.

15.3 Vérifications

tests/unitaires/test_construire_table.R compare l'empreinte d'un petit data.frame, colonnes facteur et chaîne comprises, à une valeur calculée hors de R (texte écrit par printf, empreinte par md5sum de GNU coreutils). tests/test_reproductibilite.R vérifie que l'empreinte est la même dans des processus R lancés sous LANG=C et sous LANG=C.UTF-8, égale à cette valeur, et que le rejeu de la fiche redonne l'objet.

15.4 Fonctions du module

Table 20 : Définitions de premier niveau de R/rapport.R.

Nom	Statut	Entrées	Sortie et rôle
VERSION_FICHE	constante	—	version de la fiche (1)
ENTETE_CANONIQUE	constante	—	"tablexp canonique 1"
ALGORITHME_EMPREINTE	constante	—	"md5"
STATUT_EMPREINTES_NON_CALCULEES	constante	—	statut des empreintes du résultat et du code
octets_chaines()	interne	chaînes, rôle	octets canoniques des chaînes ; refus de codage
octets_lignes()	interne	textes ASCII	octets, une valeur par ligne
colonne_canonique()	interne	colonne, rôle	libellé de type et octets ; refus type_non_canonique

Nom	Statut	Entrées	Sortie et rôle
texte_canonique()	interne	données, rôles, fichier	représentation canonique écrite en binaire
empreinte_donnees()	interne	données, rôles	MD5 de la représentation canonique
controler_provenance()	interne	provenance	refus provenance_invalide
empreintes_construction()	interne	données, rôles, provenance	champ empreintes de la fiche
assembler_fiche()	interne	rôles, plan, recette, étapes, avertissements, empreintes, provenance	fiche de construction
informations_execution()	interne	instant de début	champ execution, hors comparaison

PARTIE XII

Module simulation

Simulateur : R/simulation.R

Le dépôt ne contient aucune donnée réelle : tous les jeux sont simulés sous une *loi vraie* connue et une graine (E1.5). Le simulateur est le modèle multi-états exécuté en avant (E2.19 ; ADR 0003, point 9) : il produit des lignes individuelles portant les rôles du catalogue, sur lesquelles le moteur est vérifié. Il reproduit le même modèle que le moteur, pas le même code : il n'appelle que `donnees` et `outils`, et aucun module du moteur ne l'appelle (section 1.3). La version de cette branche est *minimale* : deux transitions concurrentes, vivant → décédé et en cours → racheté, une tête par contrat, troncature à gauche et censure par la période. Les autres formes de données (agrégats, stocks successifs), les têtes multi-contrats et l'injection de défauts paramétrés viendront avec une branche ultérieure (`docs/feuille-de-route.md`, § 2). Son point d'entrée est `simuler_portefeuille(loi, portefeuille, periode, graine)`, exporté (ADR 0007, point 3.2). Le contenu de fond vient de la spécification d'`actuary` (issue #11, partie A.2) et de son erratum du 17/10/2026.

16.1 Loi vraie et tables synthétiques

La loi vraie est une liste de deux lois, `deces` et `rachat` ; chacune associe une table de référence (classe `table_reference`, section 7.5) et un facteur par sexe, `c(H = , F = )`, réel fini ≥ 0. Pour un sexe s, la loi effective est

$$q_x^s = \min\bigl(f_s\, q_x,\ 1\bigr),$$

où q_x est lu dans la table du sexe si elle y figure, sinon dans la table `ensemble` (`loi_effective()`). Une table prospective (colonne `annee`) est refusée par le simulateur minimal. Seuls les sexes effectivement tirés (proportion d'hommes non nulle pour `H`, différente de 1 pour `F`) sont construits et contrôlés ; ils sont consignés dans `sexes_controles`.

Deux tables synthétiques, de statut `synthetique` et porteuses de leurs paramètres (attribut `parametres`), sont construites par le module :

— **Gompertz-Makeham** (`table_gompertz_makeham()`) : intensité $\mu(x) = A + B\,c^x$, avec $A \geq 0$, $B \geq 0$, $c > 1$ déclarés, sans valeur par défaut, sur des âges entiers déclarés ; la probabilité annuelle est l'intégrale exacte de μ sur $[x, x+1)$:

$$q_x = 1 - \exp\Bigl(-A - B\,c^x\,\frac{c-1}{\ln c}\Bigr)\ ;$$

la table est sans distinction de sexe (`ensemble`). Les tests emploient $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$,

$c = 1{,}1$, âges 0 à 120 (*choix de conception* de la spécification, sans prétention de réalisme), qui donnent $q_{60} = 0{,}008153179142$ et $q_{100} = 0{,}303486068312$, et des facteurs $f_H = 1$, $f_F = 0{,}6$;

— **table constante** (`table_constante()`) : $q_x = q$ à tous les âges déclarés ; elle porte la loi de rachat ($q = 0{,}05$ dans les tests de portefeuille), ou une transition absente ($q = 0$). Une loi de rachat par ancienneté (axe secondaire) est hors du périmètre de la branche.

Hypothèse de répartition dans l'année

Le simulateur suppose une **force constante par âge entier**, $\mu_x = -\ln(1 - q_x)$, dans l'échelle des années d'âge (une année d'âge vaut 1, quelle que soit sa durée de 365 ou 366 jours ; section 8.1), et des **risques concurrents indépendants**. L'hypothèse est consignée dans le résultat (`HYPOTHESE_REPARTITION`). Elle est retenue pour deux raisons :

— sous cette hypothèse, $q_x = 1 - e^{-\mu_x}$ *exactement* ; le décompte des événements rapporté à l'exposition centrale est alors l'estimateur du maximum de vraisemblance de μ_x du modèle à force constante par âge entier, avec troncature à gauche et censure indépendante, pour une observation en temps continu (résultat exact), et la cible des estimateurs à venir est connue exactement. Sur des dates au jour (section 7.1), l'exposition est arrêtée au début du jour de l'événement, d'où un écart relatif d'ordre $\mu_x/(2L_x)$, négligeable mais non nul ;

— la force constante est stable par addition des forces : une force constante dans chaque décrément équivaut à une force constante dans la table à décréments multiples, alors que la répartition uniforme n'a pas cette propriété. Deux forces indépendantes s'additionnent, et la probabilité de décès en présence du rachat a une forme close :

$$q_x^{(d)} = \frac{\mu_x^d}{\mu_x^d + \mu_x^r}\left(1 - e^{-(\mu_x^d + \mu_x^r)}\right) ;$$

sous l'hypothèse de répartition uniforme dans chaque table à décrément simple, de taux q'^d et q'^r, la survie à double décrément $(1 - t\,q'^d)(1 - t\,q'^r)$ est quadratique en t dès que les deux taux sont strictement positifs, et la table à décréments multiples n'est plus uniforme : l'hypothèse ne serait plus la même pour les deux transitions (résultat exact, calcul direct).

Une probabilité $q_x = 1$ à un âge atteignable rend la force infinie : le simulateur la refuse (`qx_egal_a_un`).

16.2 Portefeuille simulé

Entrées

Le portefeuille est une liste dont tous les champs sont requis, sans valeur par défaut (`CHAMPS_PORTEFEUILLE`), selon deux modes :

— `uniforme` : `n` contrats, `proportion_hommes`, `age_min`, `age_max`, `origine` (date au plus égale au début de la période) ; le jour d'entrée est uniforme discret sur les jours de [origine, fin], l'âge à l'entrée uniforme continu sur [age_min, age_max), et la naissance est l'entrée moins $\mathrm{round}(\text{âge} \times 365{,}25)$ jours ; l'âge exact est ensuite recalculé depuis les dates, selon la convention d'âge du moteur (section 7.1) ;

— `age_exact` : `n`, `proportion_hommes`, `age` (entier), `date_entree` ; toutes les têtes entrent à l'âge exact `age`, née le même jour et le même mois, `age` années avant l'entrée. Ce mode sert au constat de simulation (section 16.3). Une entrée le 29 février est refusée (`entree_29_fevrier`) : la naissance « `age` années avant » serait ambiguë.

Pour garantir une naissance strictement antérieure à l'entrée, il faut age_min $\times$ 365,25 ≥ 1 (la fonction `round()` de R arrondit au pair : $\mathrm{round}(0{,}5) = 0$) en mode uniforme, et un âge entier au moins égal à 1

en mode âge exact ; ces conditions sont contrôlées (`portefeuille_invalide`). La période d'observation est une liste `debut`, `fin` de dates ; la censure a lieu à l'instant $B = \text{fin} + 1$ (le simulateur n'a pas de délai de déclaration).

Algorithme

1. Pour chaque transition et chaque tête, un **temps latent** est tiré par inversion du risque cumulé H, linéaire par morceaux sur l'âge (force constante par âge entier) : l'âge de l'événement est celui où H atteint $H(a_0) + E$, avec a_0 l'âge exact à l'entrée et $E \sim \mathrm{Exp}(1)$, cherché par `findInterval()` (`temps_latent()`).
2. La sortie est le **minimum** des deux temps latents et de la censure à B, comparés en temps continu ; une égalité, de probabilité nulle, est attribuée au décès.
3. L'instant de l'événement, dans l'année d'âge k à la fraction φ, est converti en jour : $\mathrm{anniv}(k) + \lfloor \varphi\, L_k \rfloor$, avec L_k la durée de l'année d'âge, borné à $[\text{entrée}, \mathrm{anniv}(k+1) - 1]$. Le jour reste dans l'année d'âge : **la probabilité d'un événement dans l'année k est conservée exactement** ; la discrétisation n'affecte que la répartition à l'intérieur de l'année, et la borne ne corrige que l'arrondi de la virgule flottante.

Ordre des tirages et graine

Les tirages ont lieu sous `sous_graine()`, avec la graine reçue en argument, sans défaut, sous `GENERATEUR` (section 3.3), dans un ordre fixé, qui fait partie du contrat de reproductibilité et est consigné (`ORDRE_TIRAGES`) : (1) sexe, `runif(n)` comparé à la proportion d'hommes ; (2) jour d'entrée, `sample.int()` avec remise ; (3) âge à l'entrée, `runif(n)` ; (4) décès, `rexp(n)` ; (5) rachat, `rexp(n)`. En mode âge exact, (2) et (3) ne sont pas tirés. Changer cet ordre change tout jeu simulé : c'est un changement de résultat.

Couverture de la table : aucune extrapolation

Le simulateur refuse (`table_trop_courte`) une loi qui ne couvre pas tous les âges atteignables, *avant tout tirage*, si bien que le refus ne dépend pas de la graine (`controler_portefeuille()`, `controler_couverture()`). Les âges atteignables sont bornés a priori par les seuls paramètres :

— **borne basse** : l'âge révolu à l'entrée est au moins

$$y_{\min} = \max\{y : 365\,y + \lceil y/4 \rceil \le \mathrm{round}(\text{age_min} \times 365{,}25)\}$$

(`age_revolu_minimal()`), parce que y années comptent au plus $\lceil y/4 \rceil$ jours de 29 février ; la borne est prudente, les années séculaires non bissextiles ne faisant que retirer des jours. Elle donne $y_{\min} = 20$ pour age_min $= 20$ comme pour 21 : la borne $\lfloor \text{age_min} \rfloor$ serait fausse, une naissance arrondie au jour pouvant donner l'âge révolu 20 à une tête entrée à 21 ans (erratum A.2, point 4). En mode âge exact, la borne basse est l'âge déclaré ;

— **borne haute** : age_max $+$ $(B -$ origine$)/365{,}25 + 1$ en mode uniforme, et age $+$ $(B -$ date_entree$)/365{,}25 + 1$ en mode âge exact. *Preuve.* Soit $\mathrm{span}(y)$ le nombre de jours de la naissance à l'anniversaire y. Pour $y \le 130$ (âges des tables des tests), $\mathrm{span}(y) \ge 365{,}25\,y - 2{,}75$ et $\mathrm{span}(y) \le 365{,}25\,y + 0{,}75$; une tête d'âge révolu y à B vérifie $\mathrm{span}(y) \le B - n$, et, en mode uniforme, $B - n \le (B - \text{origine}) + \text{age_max} \times 365{,}25 + 0{,}5$; donc $y \le \text{age_max} + (B - \text{origine})/365{,}25 + 3{,}25/365{,}25$, strictement inférieur à la borne. En mode âge exact, $B - n = (B - \text{date_entree}) + \mathrm{span}(\text{age})$ et le même raisonnement donne une marge de $3{,}5/365{,}25$. *Statut des deux encadrements de* span : ils sont établis par calcul exhaustif, toutes naissances du 1er janvier 1700 au 31 décembre 2150 et $1 \le y \le 130$

(minimum $-2{,}75$, maximum $+0{,}75$, remesurés sur R 4.3.3 avec les fonctions de dates du module) ; le calendrier grégorien étant périodique de 400 ans, cette plage contient un cycle complet de dates de naissance, et les encadrements valent pour toute naissance (résultat exact). Au-delà de 130 ans, l'encadrement bas mesuré jusqu'à 260 ans est $-3{,}75$ (atteint pour $y = 207$, naissance au 2096-03-01), l'encadrement haut restant $+0{,}75$; la même preuve donne alors une marge de $4{,}25/365{,}25$ en mode uniforme ($4{,}5/365{,}25$ en mode âge exact), inférieure à un an pour $y \leq 260$.

Après les tirages, une assertion interne vérifie que les âges parcourus restent dans ces bornes : un écart serait une erreur de programmation, jamais un refus dépendant de la graine. Avec le portefeuille des tests (age_max $= 75$, origine 2005-01-01, fin 2024-12-31), la borne haute vaut 96 (erratum A.2, point 1), couverte par la table de 0 à 120 ans.

Sortie

`simuler_portefeuille()` rend un `data.frame` de colonnes `id_contrat`, `id_tete` (égal à `id_contrat`), `sexe`, `date_naissance`, `date_entree`, `date_sortie` (manquante si la tête est en cours à la fin) et `cause_sortie` (`deces`, `rachat` ou `en_cours`), qui portent les noms des rôles du catalogue. Toutes les lignes sont conservées, y compris celles sorties avant le début de la période, qui exercent la troncature à gauche. Les lignes sont cohérentes au sens exact, naissance $<$ entrée $\leq$ sortie $\leq$ fin, et passent les contrôles du module `donnees` sans exclusion, pour les deux transitions (constat des tests).

L'attribut `simulation` consigne : la loi vraie (tables, facteurs, loi effective par transition et par sexe avec q_x et μ_x, transitions, hypothèse de répartition, `sexes_controles`) ; le portefeuille et la période reçus ; la graine, `GENERATEUR` et l'ordre des tirages ; la version de R ; des décomptes (lignes, décès, rachats, en cours, sorties et entrées antérieures au début de la période).

Refus du simulateur : `loi_invalide`, `portefeuille_invalide`, `periode_invalide`, `entree_29_fevrier`, `table_trop_courte` et `qx_egal_a_un` ; et, par `sous_graine()`, `graine_invalide`.

16.3 Constats de simulation

Retrouver la loi vraie est une batterie de tests, chacun un *constat de simulation* qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (E2.19, E2.20). Le constat du simulateur minimal porte sur cinq cohortes de $n = 10^6$ têtes, entrées le 2016-03-15 à l'âge exact x (mode `age_exact`), période du 2016-03-15 au 2017-12-31 : l'année d'âge x est observée entière, sans censure par la période. On compte les événements D datés avant l'anniversaire $x + 1$. Sous le modèle simulé, D suit exactement la loi binomiale $\mathcal{B}(n, p)$, p étant la probabilité de l'événement dans l'année.

— **Région d'acceptation** : bilatérale exacte, $[\text{qbinom}(\alpha_k/2, n, p), \text{qbinom}(1 - \alpha_k/2, n, p)]$.

— **Correction multiple** : Bonferroni, $\alpha = 10^{-3}$ global, $K = 5$, $\alpha_k = 2 \cdot 10^{-4}$.

— **Fausse alarme exacte** de chaque test : $FA_k = P(D < \text{lo}) + P(D > \text{hi})$, calculée et écrite par le test.

— **Fausse alarme familiale** : elle est bornée par $\sum_k FA_k = 9{,}857 \cdot 10^{-4} \leq 10^{-3}$ (inégalité de Bonferroni, valable *sans hypothèse d'indépendance* ; c'est le critère vérifié). Sous indépendance des cinq cohortes, tirées sous des graines distinctes (20261016 à 20261020), elle vaut $1 - \prod_k (1 - FA_k) = 9{,}853 \cdot 10^{-4}$; cette indépendance est l'hypothèse usuelle sur le générateur, non démontrée, et la valeur n'est donnée qu'à titre d'information.

Table 21 : Constat de simulation du simulateur minimal : probabilité exacte p sous le modèle, région d'acceptation et fausse alarme exacte (tests/unitaires/test_simulation.R).

Cohorte	p	Région	FA_k
1. décès, $x = 60$, H, sans rachat	0,008153179142	[7 821 ; 8 490]	$1{,}951 \cdot 10^{-4}$
2. décès, $x = 80$, F (facteur 0,6), sans rachat	0,03150085056	[30 853 ; 32 152]	$1{,}981 \cdot 10^{-4}$
3. décès, $x = 100$, F, sans rachat	0,182091641	[180 658 ; 183 528]	$1{,}995 \cdot 10^{-4}$
4. rachat, $x = 40$, $q^r = 0{,}05$, sans décès	0,05	[49 191 ; 50 812]	$1{,}983 \cdot 10^{-4}$
5. décès, $x = 60$, H, rachat concurrent $q^r = 0{,}30$	0,006859320306	[6 554 ; 7 168]	$1{,}947 \cdot 10^{-4}$

La probabilité du test 5 est la forme close $q^{(d)}$ ci-dessus ; le rachat concurrent y est porté à 0,30 pour que le test soit sensible à son oubli (choix de la spécification). La graine étant fixe, chaque test est déterministe : sa fausse alarme est la probabilité, sur les graines, qu'une implémentation juste échoue. Les valeurs observées de D sont écrites par la batterie à chaque exécution ; elles ne sont pas reproduites ici (E3.4).

Statuts et sources d'erreur (E2.20).

— *Résultats exacts* : la formule de q_x de Gompertz-Makeham (intégrale de μ), la forme close $q^{(d)}$, la loi binomiale de D sous le modèle, les régions d'acceptation et les FA_k (quantiles et probabilités de la loi binomiale), l'inégalité de Bonferroni, la fausse alarme familiale $1 - \prod_k (1 - FA_k)$, sous l'hypothèse d'indépendance des cohortes (hypothèse sur le générateur, non démontrée), la conservation de la probabilité annuelle par la conversion en jours, les bornes de couverture.

— *Résultat asymptotique* : aucun dans ce constat.

— *Approximations numériques* : l'évaluation en virgule flottante de q_x, de μ_x et des temps latents, des quantiles binomiaux et des FA_k ; le critère $\sum_k FA_k \leq 10^{-3}$ est vérifié sur les valeurs évaluées.

— *Constat de simulation* : le passage des cinq tests.

— *Erreur Monte-Carlo* : la dispersion binomiale de D autour de np, due au nombre fini de têtes simulées ; c'est elle que mesure la largeur de la région. *Erreur d'estimation* : elle n'intervient pas ici, D étant comparé à la probabilité exacte du modèle, sans taux estimé sur une exposition ; elle viendra avec les taux bruts.

16.4 Vérifications

tests/unitaires/test_simulation.R vérifie : les valeurs des tables synthétiques (valeurs de la spécification, et recoupement par stats::integrate(), que le simulateur n'emploie pas) ; les dates d'anniversaire, 29 février compris (recoupement par as.Date() ; comparaison à age_revolu() du module donnees, écrite indépendamment) ; la cohérence exacte des lignes et l'âge révolu à l'entrée ; le passage du module donnees sans exclusion, pour les deux transitions ; l'identité des jeux à graine égale et la restauration de l'état du générateur de l'appelant ; chaque refus, dont table_trop_courte indépendant de la graine ; le constat ci-dessus. tests/test_reproductibilite.R vérifie qu'un jeu simulé est identique, attribut compris, dans un autre processus R.

16.5 Fonctions du module

Table 22 : Définitions de premier niveau de R/simulation.R.

Nom	Statut	Entrées	Sortie et rôle
CAUSES_SIMULEES	constante	—	causes produites et transitions associées
HYPOTHESE_REPARTITION	constante	—	hypothèse consignée : force constante par âge entier, risques indépendants

Nom	Statut	Entrées	Sortie et rôle
ORDRE_TIRAGES	constante	—	ordre des tirages, consigné
CHAMPS_PORTEFEUILLE	constante	—	champs requis du portefeuille, par mode
table_gompertz_makeham()	interne	A, B, c, âges	table synthétique de Gompertz-Makeham
table_constante()	interne	q, âges, nom	table synthétique à q_x constant
loi_effective()	interne	loi d'une transition, cause, sexes	q_x^s et μ_x par sexe
jours_civils()	interne	année, mois, jour	jours depuis 1970-01-01 (arithmétique entière)
jour_anniversaire()	interne	composantes de naissance, âge	jour de l'anniversaire (29 février : 1er mars des années non bissextiles)
composantes_distinctes()	interne	jours	composantes de date, calculées sur les valeurs distinctes
age_anniversaire()	interne	composantes, année, jour	âge au dernier anniversaire
date_unique()	interne	valeur	date unique entière et finie
controler_portefeuille()	interne	portefeuille, période	refus des entrées ; bornes d'âge a priori
age_revolu_minimal()	interne	écart minimal en jours	borne basse $y_{\min}$
controler_couverture()	interne	loi effective, bornes, sexes	refus table_trop_courte, qx_egal_a_un
temps_latent()	interne	loi, composantes, âge et fraction à l'entrée, entrée, tirages E	instant et jour de l'événement latent
simuler_portefeuille()	exportée	loi, portefeuille, période, graine	jeu simulé et attribut simulation

Partie XIII

Index et traçabilité

Index et traçabilité

17.1 Index des fonctions exportées

Le tableau 23 recense les fonctions exportées de `tablexp` (directives `export` de `NAMESPACE` ; ADR 0007, point 3.4) : les points d'entrée et les fonctions que l'application appelle. Les autres fonctions sont internes ; elles figurent dans le tableau des fonctions de leur module.

Table 23 : Fonctions exportées de `tablexp`.

Fonction	Module	Rôle	Section
`catalogue_roles()`	`donnees`	catalogue fermé des rôles, lu par l'application pour la déclaration des rôles	7.2
`table_reference()`	`donnees`	constructeur contrôlé d'une table de référence, appelé par l'application pour l'import de tables	7.5
`simuler_portefeuille()`	`simulation`	point d'entrée du simulateur : jeu simulé sous une loi vraie et une graine	16
`valider_plan()`	`construire_table`	validateur du plan, contrôle de saisie que l'application appellera (E4.3)	6.1
`construire_table()`	`construire_table`	point d'entrée unique : exécute un plan de construction et retourne l'objet `construction`	6

17.2 Traçabilité des exigences

Le tableau 24 relie les exigences du cahier des charges (`docs/exigences.md`) aux sections qui les décrivent. Une exigence qui n'y figure pas n'est pas encore mise en œuvre (méthodes d'estimation, de lissage, de positionnement, de fermeture et de validation, rapport figé, interface, tables de référence embarquées) ; le tableau est complété avec chaque partie rédigée.

Table 24 : Exigences et sections qui les décrivent.

Exigence	Objet	Section
E1.1	construction d'une table d'expérience, estimation d'une transition	Objet du document ; 6
E1.3	rigueur, traçabilité, reproductibilité	3, 4, 15.1
E1.5	aucune donnée réelle : jeux simulés sous une loi vraie et une graine	16

Exigence	Objet	Section
E2.2	catalogue fermé des rôles, lecture d'une colonne par son rôle	7.2
E2.3	recette la plus fine, nommée, refus si aucune ; convention d'âge, forçage plus grossier ; conventions de temps communes	7.1, 8.1
E2.4	unité d'observation, fusion par tête, incohérences entre contrats	7.4
E2.5	période déclarée, délai de déclaration	7.1, 8.2
E2.6	troncature à gauche, censure à droite, censure non informative nommée	8.2
E2.7	refus par défaut, motifs en liste fermée, exclusions déclarées, données inchangées	7.3
E2.8	pondération par les montants : rôle refusé tant que ses intervalles ne sont pas spécifiés	7.2
E2.9	segmentation déclarée, vide explicite admis (le test d'homogénéité viendra avec la validation)	6.1, 8.2
E2.11	noyau multi-états : une table estime une transition ; seconde transition couverte par les tests	6.1, 16
E2.19	simulateur : loi vraie, entrées, sorties concurrentes, censure, constat de simulation	16
E2.20	quatre statuts, erreur Monte-Carlo et erreur d'estimation	3.1, 8.1, 16.3
E3.1	document LaTeX unique, une partie par module, charte d'`outil_USP`, tableaux de fonctions	ensemble du document
E3.3	test de concordance entre la documentation et le code	4.1
E3.4	aucun chiffre issu d'une exécution	Objet du document
E3.7	relecture sans le code, rejeu à partir de la fiche	15.1
E4.1	R 4.3.3, package `tablexp`, point d'entrée unique, modules	1.2, 6
E4.2	packages de priorité `base` ; aucune dépendance à Java	5.1, 5.2
E4.3	frontière calcul / affichage ; contrôles de saisie dans le package	1.1, 6.1
E4.4	le moteur reçoit un `data.frame` et des rôles, refuse un type incompatible	1.1, 7.2
E4.5	plan exécuté d'un bloc, objet complet, contenu de la fiche	6, 15.1
E4.6	reproductibilité, graine locale, générateur fixé, aucun horodatage comparé	3, 6.3
E4.7	références de non-régression sur la plateforme de référence	4.3
E4.9	un module par famille, index des fonctions contrôlé par la concordance	1, 17.1
E5.5	messages qui disent ce qui manque, sans rien corriger	2.1, 7.3

17.3 Références

— M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`) ; section 3.2.

— R Core Team, *R : A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienne, 2024 (`https://www.R-project.org/` ; référence rendue par `citation()` sous R 4.3.3).